



FORMATION CONTROL OF AGENTS UNDER DYNAMIC METEOROLOGICAL CONDITIONS

KON 4902E CONTROL & AUTOMATION ENGINEERING DESIGN II

Interim Report

Author

ATAKAN YAMAN

SID: 110190235

Supervised by

DR. ABDURRAHMAN YILMAZ

Submitted in fulfillment of requirements for the degree of
Bachelor of Science in Control & Automation Engineering

Department of Control and Automation Engineering
Istanbul Technical University
2025

*"A soul in tension that's learning to fly
Condition grounded but determined to try
Can't keep my eyes from the circling skies
Tongue-tied and twisted, just an earthbound misfit, I."*

Pink Floyd, 1987

"...to those who find order in the chaos of the wind."

Abstract

This study designs and validates a formation control framework for robotic agents that adapt to dynamic weather conditions. Motivated by use cases such as post-disaster search-and-rescue, autonomous patrol, and environmental monitoring, the system aims to adaptively maintain formation stability in the presence of real or simulated weather disturbances (e.g., wind gusts).

To achieve this, a multi-layer control architecture is proposed. At the core lies a conventional Proportional-Integral-Derivative (PID) controller for basic trajectory tracking, augmented by an Interval Type-2 Fuzzy Logic System (IT2-FLS) to enhance robustness under environmental uncertainty and sensor noise. In the current implementation, coordination among agents is managed by a Centralized supervisor node to ensure geometric consistency, establishing a robust baseline for future distributed mechanisms such as Particle Swarm Optimization (PSO).

The proposed system is implemented as modular Robot Operating System 2 (ROS2) nodes. The controllers are validated within the Gazebo physics simulation environment, which provides high-fidelity rigid-body dynamics and wind force modeling. This setup serves as a verification stage before extending the architecture to other platforms like AirSim for advanced meteorological scenarios.

The performance of the proposed approach is benchmarked against baseline PID controllers using criteria such as formation tracking error and recovery time after gust events. While focusing on centralized coordination in this phase, the modular design paves the way for future integration of Event-Triggered Communication (ETC) and distributed consensus algorithms. All source code and simulation data are published via GitHub to ensure reproducibility.

Keywords : Formation control, Interval Type-2 Fuzzy Logic, Centralized Coordination, PID controller, ROS2, Gazebo Simulation, Wind disturbance rejection.

Abstract in Turkish(Özet)

Bu çalışma, dinamik hava koşullarına uyum sağlayan robotik ajanlar için bir formasyon kontrol çerçevesi tasarlamakta ve doğrulamaktadır. Afet sonrası arama-kurtarma, otonom devriye ve çevresel izleme gibi kullanım senaryolarından yola çıkılan bu sistem, gerçek veya simüle edilmiş meteorolojik bozucular (örneğin, rüzgar hamleleri) varlığında formasyon kararlılığını uyarlamalı olarak sürdürmeyi amaçlamaktadır.

Bu amaca ulaşmak için çok katmanlı bir kontrol mimarisi önerilmektedir. Mimarinin merkezinde temel yörünge takibi için geleneksel bir PID denetleyici yer alırken, çevresel belirsizlik ve sensör gürültüsü altında gürbüzlüğü artırmak için bir Aralık Tip-2 Bulanık Mantık Sistemi (IT2-FLS) ile güçlendirilmiştir. Mevcut uygulamada, ajanlar arasındaki koordinasyon, geometrik tutarlılığı sağlamak amacıyla Merkezi bir yönetici düğüm tarafından yönetilmekte olup, bu yapı PSO gibi gelecekteki dağıtık mekanizmalar için sağlam bir temel oluşturmaktadır.

Önerilen sistem, modüler ROS2 düğümleri olarak uygulanmıştır. Denetleyiciler, yüksek sadakatli katı cisim dinamiği ve rüzgar kuvveti modellemesi sağlayan Gazebo fizik simülasyon ortamında doğrulanmaktadır. Bu kurulum, mimariyi gelişmiş meteorolojik senaryolar için AirSim gibi diğer platformlara genişletmeden önce bir doğrulama aşaması görevi görmektedir.

Önerilen yaklaşımın performansı, formasyon takip hatası ve rüzgar hamlesi olaylarından sonra toparlanma süresi gibi kriterler kullanılarak temel PID denetleyicilere karşı kıyaslanmıştır. Bu aşamada merkezi koordinasyona odaklanılsa da, modüler tasarım gelecekte ETC ve dağıtık uzlaşım algoritmalarının entegrasyonuna zemin hazırlamaktadır. Tekrarlanabilirlik adına tüm kaynak kodları ve simülasyon verileri GitHub üzerinden yayınlanmıştır.

Anahtar Kelimeler : Formasyon kontrolü, Aralık Tip-2 Bulanık Mantık, Merkezi Koordinasyon, PID denetleyici, ROS2, Gazebo Simülasyonu, Rüzgar bozucu bastırma.

Declaration of Originality

We, the undersigned students, declare that this report was entirely prepared through our own efforts and work. All sections of the report that rely on external sources have been explicitly stated and properly cited.

During the preparation of the report, AI tools were used solely for supporting tasks such as grammar corrections, writing adjustments, and formatting. No AI tools or external sources were used for content generation in any part of the report.

For projects involving teamwork, we confirm that the contribution percentages specified in the Individual Contribution Statement section at the end of the report are accurate and that all team members contributed to the project in alignment with these percentages.

The checklist accompanying this report has been accurately and completely filled out in accordance with the relevant instructions.

Signatures:

Name and Surname	Student ID	Date	Signature
Atakan Yaman	110190235		

Copyright Declaration

The copyright of this thesis rests with the author. No quotation from it should be published without the author's prior written consent and information derived from it should be acknowledged.

I hereby grant to Istanbul Technical University the right to archive and to make available my thesis or dissertation in whole or in part in the University Libraries in all forms of media, now or hereafter known, subject to the provisions of the Copyright Act. I retain all proprietary rights, such as patent rights. I also retain the right to use in future works (such as articles or books) all or part of this thesis or dissertation.

Acknowledgments

I would like to express my sincere gratitude to my advisor, Dr. Abdurrahman YILMAZ, for his invaluable guidance, continuous support, and technical insights throughout this project. His mentorship has been instrumental in shaping the direction of this study.

I also extend my thanks to the faculty members of the Control and Automation Engineering Department for providing a stimulating academic environment.

Finally, I would like to thank my family and friends for their encouragement and patience during this intense study period.

Contents

Dedication	i
Abstract	iii
Abstract in Turkish(Özet)	v
Declaration of Originality	vii
Copyright Declaration	ix
Acknowledgments	xi
List of Acronyms	xvii
List of Symbols	xix
List of Figures	xxi
List of Tables	xxiii
1 Introduction	1
1.1 Motivation	1
1.2 Design Criteria	2
1.3 Literature Review	4
1.4 Contribution of the Study	4
1.5 Structure of the Report	5
2 Literature Review	7
2.1 Review existing research and related works in the field	7
2.2 Identify gaps and limitations in previous studies	8
2.3 Justify the need for the proposed approach	8
3 Methodology	11
3.1 System Architecture and Scope	11
3.1.1 Centralized Coordination Strategy	11
3.1.2 Planar Dynamics Assumption	12

3.2	Theoretical Foundations and Mathematical Models	12
3.2.1	Drone Dynamics Model	12
3.2.2	Wind Field Modeling	13
3.3	Proposed Method and Controller Design	15
3.3.1	PID Controller Mathematical Foundation	15
3.3.2	Interval Type-2 Fuzzy Logic System (IT2-FLS)	16
3.3.3	FLS Design Visualization	17
3.4	Tools, Software, and Experimental Setup	17
3.4.1	Simulation Environment	17
3.4.2	Automation and Configuration	17
3.4.3	Data Analysis Pipeline	19
4	Implementation	21
4.1	Simulation Environment	21
4.2	Practical Realization of the Proposed Method	21
4.3	System Architecture and Hardware/Software Implementation	22
4.4	Challenges and Solutions	25
5	Simulations, Results and Discussion	27
5.1	Phase 1: C++ Standalone Algorithm Validation	28
5.1.1	Comparative Performance Analysis (Standalone)	28
5.1.2	Internal Controller Dynamics (Standalone)	28
5.1.3	Overall Formation Trajectory (Standalone)	31
5.1.4	Quantitative Comparison (Standalone Results)	31
5.2	Phase 2: ROS 2 Integration and Experimental Results	32
5.2.1	Experimental Setup and Scenarios	32
5.2.2	Formation Establishment and Transient Response	32
5.2.3	Disturbance Rejection Performance and Controller Comparison	34
5.2.4	Energy Efficiency and System Health	36
5.2.5	Multi-Agent Consistency	38
5.2.6	Quantitative Performance Metrics (ROS 2)	40
5.3	Discussion and Key Findings	41
6	Conclusion and Future Work	43
6.1	Future Work	44
6.2	Impact of the Study	44

Ethical Rules Compliance Statement	45
Individual Contribution Statement	49
Constraints and Engineering Standards Used in the Report	51
A Implementation	55
A.1 Practical Realization of the Proposed Method	55
A.2 System Architecture and Hardware/Software Implementation	56
A.3 Challenges and Solutions	57
A Additional Simulation Data	59
A.1 Drone 1 (Left Wing) Analysis	59
A.2 Drone 2 (Right Wing) Analysis	61
B Implementation Details	63
B.1 Control Parameters	63
B.2 Additional Simulation Views	64
References	65
Resume 1	67

List of Acronyms

ETC Event-Triggered Communication

GT2-FLS General Type-2 Fuzzy Logic System

PID Proportional-Integral-Derivative

PSO Particle Swarm Optimization

ROS2 Robot Operating System 2

UAV Unmanned Aerial Vehicle

List of Symbols

$a(t)$	Agent acceleration vector	m s^{-2}
C_{lin}	Linear drag coefficient	kg s^{-1}
C_{quad}	Quadratic drag coefficient	kg m^{-1}
$e(k)$	Tracking error at time step k	m
F_{drag}	Aerodynamic drag force	N
F_{thrust}	Motor thrust force	N
F_{wind}	Wind disturbance force vector	N
g	Gravitational acceleration	m s^{-2}
K_p, K_i, K_d	PID controller gains	
$K_{p,z}, K_{d,z}$	Altitude hold P-D gains	
m	Mass of the agent	kg
$p(t)$	Agent position vector	m
$r(k)$	Reference setpoint	m
$u(k)$	Control input signal	m s^{-2}
v_{rel}	Relative airspeed	m s^{-1}
v_{wind}	Wind velocity vector	m s^{-1}
w	Drag weighting function	
z_{ref}	Reference altitude	m
α	Anti-windup / Filter coefficient	
Δt	Simulation time step	s
μ	Fuzzy membership grade	
τ	Motor response time constant	s
$\tilde{A}$	Type-2 Fuzzy Set	

List of Figures

3.1	Interval Type-2 Fuzzy Logic System (IT2-FLS) membership functions for the three input variables: Position Error, Error Derivative (dError), and Wind. The shaded regions (Footprints of Uncertainty - FOU) represent the system's ability to handle uncertainty. . .	18
3.2	The FLS control surface illustrates the relationship between the Error and Wind inputs and the FLS Correction Output , with the dError input held constant at zero. This surface visualizes the expert knowledge encoded in the rule base.	19
4.1	ROS2 Computation Graph. The diagram shows the connectivity between the <code>wind_tool</code> , distributed agent controller nodes, and the dashboard bridge.	23
4.2	Real-time Monitoring Dashboard. This interface provides instant feedback on formation status (Active/Settled), displays aggregate error metrics for controller groups (PID_FUZZY vs PD), and visualizes the active wind vector.	24
5.1	Phase 1: Basic Performance Analysis. (Top-Left) Position error over time showing reduced peaks with FLS. (Top-Right) Final tracking accuracy comparison. (Bottom) Drone 0 tracking response and average error comparison.	29
5.2	Phase 1: Detailed Error Analysis. The plots illustrate the X-Axis and Y-Axis error breakdowns, error convergence rates, and the formation centroid tracking error.	29
5.3	Phase 1: Detailed Analysis for Drone 0. The third row compares the raw PID output (dotted) with the FLS correction (dashed), demonstrating active wind rejection against the simulated wind profile (bottom row).	30
5.4	Phase 1: Overall Formation Control Analysis. (Left) 2D Trajectory paths of the swarm maintaining triangular geometry. (Right) Position control responses and final command signals for Drone 0.	31
5.5	Gazebo Simulation Environment. Red markers indicate the target setpoints for the triangular formation structure.	33
5.6	Simulation Configuration Panel. Displays the active wind force vector and the controller assignment for each agent group.	33
5.7	2D Formation Map. Visualizes the real-time position of agents (Blue dots) versus their target coordinates (Green crosses) in the X-Y plane.	34
5.8	Position Tracking Error Over Time for all agents. The graph shows the initial formation establishment spike and subsequent convergence.	35
5.9	Zoomed View of Formation Error. After the initial transient, the error remains bounded within the safety threshold, indicating successful formation maintenance.	35
5.10	Detailed Controller Comparison (Zoomed). The Green line (Hybrid) demonstrates significantly lower oscillation amplitude compared to the Blue line (PID), validating the active damping effect of the Fuzzy Logic layer.	36
5.11	Maximum Overshoot and RMSE Analysis comparing PID, PD, and Hybrid architectures. The PD controller shows significant steady-state error.	37

5.12	Zoomed Analysis of Overshoot Spikes. The Hybrid controller consistently suppresses peak overshoot values faster than the standard PID controller.	37
5.13	Detailed Transient Response Analysis. The rapid decay of the error signal in the Hybrid system confirms the effectiveness of the fuzzy damping rules.	38
5.14	Integral Absolute Error (IAE) Comparison. The graph clearly demonstrates the inefficiency of the PD controller compared to the optimized PID and Hybrid approaches. . . .	39
5.15	Zoomed View of IAE Accumulation. The flat slope of the Hybrid controller's IAE indicates superior steady-state energy efficiency.	39
5.16	Per-Agent Maximum Overshoot and RMSE Analysis. The consistent performance across Agent 0 through Agent 8 verifies the scalability of the control approach.	40
A.1	PID+FLS Detailed Analysis for Drone 1 (Left Wing). The plots show position errors, PID term breakdowns, and final command composition.	60
A.2	PID+FLS Detailed Analysis for Drone 2 (Right Wing). Comparisons of simulated wind conditions and controller response.	61
B.1	Controller Parameter Configuration. Shows the specific K_p, K_i, K_d gains assigned to each agent group.	63
B.2	Wide-angle view of the Gazebo simulation environment during formation initialization. . .	64

List of Tables

3.1	Simulation Physics Parameters and Code Correspondence	15
3.2	Fuzzy Rule Base for the Interval Type-2 FLS	20
5.1	Quantitative Performance Comparison (C++ Standalone)	32
5.2	Quantitative Performance Comparison (ROS 2 Integration Phase)	40

1

Introduction

Contents

1.1 Motivation	1
1.2 Design Criteria	2
1.3 Literature Review	4
1.4 Contribution of the Study	4
1.5 Structure of the Report	5

MOTIVATION

- **Background and Context**
 - **Overview:** This study addresses the critical challenge of maintaining coordinated multi-agent robotic formations under dynamic and uncertain meteorological conditions.
 - **Importance:** In real-world scenarios such as post-disaster search-and-rescue, environmental monitoring, and autonomous border patrol, robotic teams must operate reliably despite harsh atmospheric disturbances like wind gusts and sudden temperature gradients.
 - **Relevance:** As autonomous systems move from controlled lab environments to the field, the ability to reject environmental disturbances becomes a prerequisite for operational safety and mission success.
- **Problem Identification**
 - **Key Challenges:** Environmental factors, particularly wind vectors and turbulence, significantly degrade the coordination among agents, impair sensor accuracy, and compromise formation integrity.

- **Limitations:** Conventional formation control methods (e.g., standard PID, Leader-Follower) often lack the adaptability required for highly dynamic environments. Recent studies indicate that while traditional strategies offer simplicity, they fail to guarantee stability under nonlinear dynamics and unknown disturbances.
- **Need for Innovation:** There is a distinct need for a hybrid control architecture that actively incorporates meteorological awareness into the control loop rather than treating weather merely as noise.

- **Significance of the Project**

- **Contribution:** This project develops a "meteorology-aware" formation control framework using **Interval Type-2 Fuzzy Logic Systems (IT2-FLS)** within a centralized coordination architecture.
- **Impact:** Successfully validating this system will contribute to the development of more resilient multi-robot formations capable of operating in critical field applications where failure is not an option.
- **Advancement:** The use of the high-fidelity **Gazebo** simulation environment ensures robust physics modeling and ROS 2 standard alignment, facilitating future real-world deployment.

- **Personal and Academic Motivation**

- **Interdisciplinary Approach:** Combining my background in **Meteorological Engineering** with **Control and Automation Engineering**, this project represents a unique intersection of atmospheric science and robotics.
- **Career Aspirations:** This study serves as a foundation for advanced research in autonomous systems operating in unstructured environments, directly supporting my goal of developing robust field robotics.
- **Skill Development:** The project leverages my expertise in ROS 2, Python/C++, and embedded systems, further refining skills in adaptive control theory and system integration.

DESIGN CRITERIA

- **Performance Requirements**

- **Tracking Accuracy:** The system must minimize formation tracking error (RMSE) to effectively maintain formation integrity. The target is to keep RMSE below 5% of the agent size or formation dimension.

- **Response Time:** The system must adapt to dynamic weather effects with a recovery time of less than 2 seconds after a significant gust event.
- **Robustness:** The controller must maintain stability without oscillation under simulated wind forces up to defined safety limits.
- **Technical Specifications**
 - **Architecture:** The system is built on a modular ROS 2 (Robot Operating System) architecture, utilizing a centralized supervisor for coordination.
 - **Algorithms:** Core control relies on a hybrid PID + **Interval Type-2 Fuzzy Logic System (IT2-FLS)**.
 - **Simulation:** Validation is performed using the **Gazebo** physics engine, which provides high-fidelity rigid-body dynamics and wind force modeling.
- **Feasibility and Constraints**
 - **Computational Load:** The control algorithms must be lightweight enough to run on standard embedded processors, avoiding heavy online optimization.
 - **Bandwidth:** Inter-agent communication is optimized to prevent network congestion, crucial for scalable formations.
- **Scalability and Flexibility**
 - **Modularity:** All modules (Control, Coordination, Communication) are implemented as independent ROS 2 nodes, allowing the system to scale or transition to distributed architectures in the future without redesign.
 - **Portability:** The software stack is designed to be hardware-agnostic, facilitating future migration from simulation to real physical robots.
- **Safety and Reliability**
 - **Collision Avoidance:** A minimum safe distance (e.g., 1 meter) must be maintained between agents at all times, even under external disturbances.
 - **Fault Tolerance:** The modular nature ensures that system components can be monitored and debugged effectively.
- **Environmental and Sustainability Considerations**
 - **Simulation-First Approach:** Prioritizing high-fidelity simulation reduces material waste and energy consumption associated with physical crash testing during the development phase.

- **Energy Efficiency:** Control strategies are optimized to minimize actuator effort (estimated <15 Watts for control operations), extending mission endurance.

LITERATURE REVIEW

See Chapter 2.

CONTRIBUTION OF THE STUDY

- **Scientific Contributions**

- Validation of a hybrid control architecture combining PID and **Interval Type-2 Fuzzy Logic (IT2-FLS)** for disturbance rejection.
- Development of a meteorology-aware formation strategy that adapts control parameters based on real-time or simulated wind data.

- **Technical Contributions**

- Implementation of a fully modular ROS 2 node structure validated in the **Gazebo** simulation environment.
- Integration of a centralized coordination mechanism that serves as a robust baseline for future distributed systems.

- **Practical and Industrial Contributions**

- A robust framework applicable to disaster relief drones and autonomous environmental monitoring systems.
- Reduction of maintenance costs and collision risks through improved stability control.

- **Academic and Educational Contributions**

- Publication of all source code, simulation models, and datasets via GitHub to support open-source research and reproducibility.
- Providing a benchmark for comparing Type-1 vs. Type-2 Fuzzy Logic performance in aerial robotics.

- **Societal and Environmental Contributions**

- Supporting safer search-and-rescue operations by ensuring drone reliability in adverse weather.
- Promoting sustainable engineering through energy-efficient control algorithms and simulation-based validation.

STRUCTURE OF THE REPORT

This report is organized to systematically present the design, development, and validation of the proposed system. A brief outline of the subsequent sections is provided below:

- **The Design Problem:** Defines the core engineering problem, detailed design criteria, and the specific technical challenges addressed by this project.
- **Development Plan:** Outlines the work packages, timeline, and progress made during the current semester (KON4901) and the plan for the next semester (KON4902).
- **Risk Plan:** Identifies potential technical and economic risks (e.g., simulation integration issues, hardware constraints) and proposes mitigation strategies and "Plan B" alternatives.
- **Constraints and Engineering Standards:** Details the safety, operational, and ethical constraints shaping the project, along with the engineering standards (IEC, ISO, IEEE) adhered to during design.
- **Experiment and Simulation Setup:** Describes the system architecture, mathematical framework (PID, **IT2-FLS**, Drone Dynamics), control algorithms, and presents the preliminary simulation results validating the approach.
- **Impact of the Study:** Discusses the broader environmental, economic, and societal implications of the project, including ethical compliance and sustainability metrics.

2

Literature Review

Contents

2.1 Review existing research and related works in the field	7
2.2 Identify gaps and limitations in previous studies	8
2.3 Justify the need for the proposed approach	8

REVIEW EXISTING RESEARCH AND RELATED WORKS IN THE FIELD

The field of multi-agent formation control has evolved significantly, transitioning from rigid geometric structures to highly adaptive, intelligent swarms. Comprehensive reviews indicate that traditional strategies, such as leader–follower and virtual structure models, offer simplicity and reliability in static environments. As highlighted in [1], these conventional formation control methods provide established frameworks with well-understood principles and less computational burden, making them the standard for industrial automation and controlled laboratory settings.

However, recent research trends have shifted towards robust adaptive coordination strategies capable of maintaining stability under nonlinear dynamics and unknown disturbances. For instance, [2] demonstrated robust formation coordination of robot swarms dealing specifically with nonlinear dynamics, proving that linear controllers are often insufficient for complex maneuvers.

Furthermore, communication-aware and decentralized approaches have gained prominence to address network constraints. [3] proposed an Edge-AI-based UAV swarm system integrating Multi-Agent Reinforcement Learning (MARL) with an efficient decentralized network structure. Their work showed adaptive decision-making capabilities and significant improvements in communication efficiency. Simi-

larly, [4] explored fixed-time distributed formation control for heterogeneous systems, emphasizing the importance of decentralized strategies in complex, multi-domain setups.

In the domain of intelligent control, Fuzzy Logic Systems (FLS) have been widely adopted to handle uncertainty. [5] introduced an event-triggered Type-3 Fuzzy Logic Control for fault-tolerant systems, pushing the boundaries of uncertainty management beyond standard Type-1 and Type-2 approaches. Additionally, comparative studies by [6] have explicitly analyzed the performance trade-offs between PID and fuzzy controllers, often favoring fuzzy approaches in non-linear scenarios.

IDENTIFY GAPS AND LIMITATIONS IN PREVIOUS STUDIES

Despite the significant advancements mentioned above, several critical limitations persist in the current literature:

- **Sensitivity to Environmental Disturbances:** While conventional methods are reliable, they often lack the adaptability required to operate in highly dynamic environments [1]. Studies such as [7] have shown that standard PID and rigid formation controllers degrade significantly when exposed to strong external disturbances like wind gusts or turbulence.
- **Weather as Noise vs. State Variable:** Although AI and fuzzy logic approaches are gaining traction, many existing implementations treat meteorological conditions merely as random noise rather than an active state variable [8]. This limits the system's ability to proactively compensate for predictable aerodynamic forces.
- **Computational Constraints:** Some robust architectures, particularly those relying on deep reinforcement learning or complex optimization, are often computationally too heavy for real-time edge deployment on small-scale drones [9]. This creates a barrier to practical implementation on hardware with limited processing power.
- **Simulation Fidelity and Portability:** Many implementations remain limited to specific, isolated simulation environments without ensuring cross-platform portability. As noted in simulator comparison studies like [10], bridging the gap between distinct simulation engines and real-world hardware remains a challenge, often hindering the transition from theoretical design to practical field deployment [11].

JUSTIFY THE NEED FOR THE PROPOSED APPROACH

To address the identified gaps, this study proposes a hybrid, meteorology-aware control architecture. The necessity of this approach is grounded in the following justifications:

1. **Handling Uncertainty with Type-2 Fuzzy Logic:** The failure of linear controllers to handle non-linear wind dynamics necessitates a more robust approach. By combining conventional PID control with General Type-2 Fuzzy Logic Systems (GT2-FLS), the proposed system enhances robustness under sensor noise and environmental uncertainty. This direction is supported by recent advances in fuzzy model predictive control, which have shown superior performance in constrained maneuver control [12].
2. **Centralized Coordination for Robustness:** While distributed methods are promising, they introduce complexity in synchronization. This study adopts a centralized approach to ensure geometric consistency and fault-free trajectory generation, serving as a solid baseline for future distributed implementations like PSO-based consensus [13].
3. **High-Fidelity Validation:** This project bridges the simulation gap by structuring the entire system as modular ROS 2 nodes. Unlike abstract mathematical models, this architecture is validated in the **Gazebo** physics engine [14], ensuring that the control logic is tested against realistic inertia, drag, and wind forces, making it ready for future hardware deployment.

3

Methodology

Contents

3.1 System Architecture and Scope	11
3.1.1 Centralized Coordination Strategy	11
3.1.2 Planar Dynamics Assumption	12
3.2 Theoretical Foundations and Mathematical Models	12
3.2.1 Drone Dynamics Model	12
3.2.2 Wind Field Modeling	13
3.3 Proposed Method and Controller Design	15
3.3.1 PID Controller Mathematical Foundation	15
3.3.2 Interval Type-2 Fuzzy Logic System (IT2-FLS)	16
3.3.3 FLS Design Visualization	17
3.4 Tools, Software, and Experimental Setup	17
3.4.1 Simulation Environment	17
3.4.2 Automation and Configuration	17
3.4.3 Data Analysis Pipeline	19

SYSTEM ARCHITECTURE AND SCOPE

Before detailing the mathematical models, it is essential to define the architectural scope and assumptions governing this study.

3.1.1 Centralized Coordination Strategy

Contrary to fully distributed approaches where agents negotiate consensus, this study employs a **centralized coordination architecture**. A single **Formation Coordinator** node acts as the mission supervisor, calculating the desired reference trajectories for the entire formation based on a pre-defined

geometric shape (e.g., triangle, line). These references are broadcast to individual agents via the Robot Operating System 2 (ROS 2) middleware. This design choice simplifies the communication overhead and allows for focused analysis of the disturbance rejection performance of the local controllers.

3

3.1.2 Planar Dynamics Assumption

While the simulation environment utilizes the 3D physics capabilities of Gazebo, the control problem is simplified to the **horizontal plane** (x, y) . Vertical stabilization is handled by an automatic altitude-hold controller embedded within the physics engine. This decoupling allows for a rigorous evaluation of the proposed Interval Type-2 Fuzzy Logic System (IT2-FLS) in handling lateral wind disturbances, which is the primary contribution of this work.

THEORETICAL FOUNDATIONS AND MATHEMATICAL MODELS

This section details the mathematical framework governing the simulation environment. To ensure high-fidelity testing of formation algorithms, the simulation employs a specialized 2D dynamics model where the vertical (z) motion is decoupled and automatically stabilized. This allows the control system to focus on planar (x, y) formation maneuvers while subject to realistic aerodynamic and actuator constraints.

3.2.1 Drone Dynamics Model

The fundamental equations of motion for the planar axes $i \in \{x, y\}$ are defined as:

$$m\ddot{p}_i(t) = F_{thrust,i}(t) + F_{drag,i}(t) + F_{wind,i}(t) \quad (3.1)$$

where m is the drone mass, p_i is the position, and the forces represent thrust, aerodynamic drag, and wind disturbances respectively.

Actuator Dynamics

Unlike ideal models, this simulation incorporates realistic motor response delays. The commanded acceleration $u_i(t)$ is passed through a first-order lag filter with asymmetric time constants for acceleration (τ_{up}) and deceleration (τ_{down}):

$$\dot{a}_i(t) = \begin{cases} \frac{1}{\tau_{up}}(u_i(t) - a_i(t)) & \text{if } |u_i| > |a_i| \\ \frac{1}{\tau_{down}}(u_i(t) - a_i(t)) & \text{otherwise} \end{cases} \quad (3.2)$$

The resulting thrust force applied to the body is $F_{thrust,i} = m \cdot a_i(t)$. This asymmetry models the physical reality that rotors spin up slower than they spin down (or vice-versa depending on ESC braking).

Aerodynamic Drag Model

The drag force F_{drag} models air resistance using a blended linear-quadratic model based on the relative airspeed $v_{rel} = v_{drone} - v_{wind}$. A weighting function $w(v_{rel})$ transitions smoothly between laminar (linear) and turbulent (quadratic) flow regimes:

$$F_{drag,i} = -((1 - w)C_{lin} + wC_{quad}|v_{rel}|)v_{rel,i} \quad (3.3)$$

where the weighting function is defined as:

$$w(v_{rel}) = \text{clamp}\left(\frac{|v_{rel}| - v_{thr}}{v_{thr}}, 0, 1\right) \quad (3.4)$$

Here, C_{lin} and C_{quad} are the linear and quadratic drag coefficients, and v_{thr} is the transition speed threshold.

3.2.2 Wind Field Modeling

To rigorously test the disturbance rejection capabilities of the proposed control architecture, a deterministic wind field model is employed. Unlike stochastic models (e.g., Dryden or Von Karman) which are suitable for long-term fatigue analysis, this study utilizes discrete gust steps to evaluate the transient response and recovery time of the formation.

The wind field is generated by an external ROS 2 node (`wind_tool.py`) and injected into the simulation environment via two synchronized topics:

- `/wind/velocity`: Represents the ambient air velocity vector $\mathbf{v}_{wind} = [v_x, v_y, v_z]^T$, affecting the aerodynamic drag calculation (Equation 3.3).

- **/wind/force:** Represents the direct force exerted by the wind on the drone body $\mathbf{F}_{wind} = [F_x, F_y, F_z]^T$.

Mathematical Formulation

The wind disturbance is modeled as a time-varying step function, allowing for precise reproducibility of "gust" events. The wind vector $\mathbf{W}(t)$ (representing either velocity or force) is defined as:

$$\mathbf{W}(t) = \begin{cases} \mathbf{0} & \text{if } t < t_{start} \\ \mathbf{A} \cdot S & \text{if } t_{start} \leq t < t_{end} \\ \mathbf{0} & \text{if } t \geq t_{end} \end{cases} \quad (3.5)$$

where:

- $\mathbf{A} = [A_x, A_y, A_z]^T$ is the base amplitude vector of the disturbance.
- S is a global scaling factor used to adjust severity without altering direction.
- $[t_{start}, t_{end}]$ defines the active duration of the gust.

This deterministic approach ensures that every experimental run (e.g., PID vs. IT2-FLS) is subjected to the exact same disturbance profile, enabling a fair and quantitative comparison of recovery performance.

Altitude Hold Controller

The vertical motion is constrained by an automatic PD controller within the physics plugin, ensuring the drone maintains a fixed altitude z_{ref} . This effectively reduces the control problem to a 2D planar tracking task:

$$F_z = m(g + K_{p,z}(z_{ref} - z) - K_{d,z}\dot{z}) \quad (3.6)$$

Simulation Parameters

The key physical parameters used in the `agent_control_pkg` core physics library are summarized in Table 3.1.

Table 3.1: Simulation Physics Parameters and Code Correspondence

Parameter	Symbol	Value	Code Variable
Drone Mass	m	1.5 kg	this->mass_
Actuator Time Const. (Up)	τ_{up}	0.15 s	actuator_tau_up
Actuator Time Const. (Down)	τ_{down}	0.10 s	actuator_tau_down
Linear Drag Coeff.	C_{lin}	0.1	drag_coeff_lin
Quadratic Drag Coeff.	C_{quad}	0.05	drag_coeff_quad
Altitude P Gain	$K_{p,z}$	12.0	altitude_kp_
Altitude D Gain	$K_{d,z}$	4.0	altitude_kd_

PROPOSED METHOD AND CONTROLLER DESIGN

The proposed control architecture utilizes a hybrid approach combining a standard PID controller with an Interval Type-2 Fuzzy Logic System (IT2-FLS) to handle uncertainties in wind measurements.

3.3.1 PID Controller Mathematical Foundation

The core PID controller implements a discrete-time control law incorporating anti-windup and derivative filtering:

$$u(k) = K_p e(k) + K_i \sum_{j=0}^k e(j) \Delta t + K_d \frac{m(k) - m(k-1)}{\Delta t} \quad (3.7)$$

To prevent integral windup during actuator saturation, a conditional integration strategy is applied:

$$I(k+1) = \begin{cases} I(k) + e(k) \Delta t & \text{if } |u(k)| < u_{\max} \\ I(k) - \alpha \cdot \text{excess_output} / K_i & \text{if output clamped} \end{cases} \quad (3.8)$$

3.3.2 Interval Type-2 Fuzzy Logic System (IT2-FLS)

To handle uncertainty in wind measurements, an IT2-FLS is utilized. The inference process involves fuzzification of crisp inputs (Error, dError, Wind) into interval Type-2 membership functions:

$$\tilde{A}_i(x) = [\underline{\mu}_{\tilde{A}_i}(x), \overline{\mu}_{\tilde{A}_i}(x)] \quad (3.9)$$

The final crisp output is obtained via the Karnik-Mendel type reduction algorithm and centroid defuzzification:

$$Y_{tr} = [y_l, y_r] = \left[\frac{\sum_{i=1}^L f_i^l y_i^l}{\sum_{i=1}^L f_i^l}, \frac{\sum_{i=1}^R f_i^r y_i^r}{\sum_{i=1}^R f_i^r} \right], \quad y = \frac{y_l + y_r}{2} \quad (3.10)$$

The effectiveness of the IT2-FLS relies heavily on its knowledge base. The specific rule base implemented in the system configuration (`fuzzy_params.yaml`) consists of 21 expert-derived rules designed to minimize tracking error while providing active damping and wind compensation. The complete rule set is presented in Table 3.2.

Legend for Linguistic Variables:

- **Error/Output:** NB (Negative Big), NS (Negative Small), ZE (Zero), PS (Positive Small), PB (Positive Big), XLNC (Extra Large Negative Correction), LNC (Large Negative), SNC (Small Negative), NC (No Correction), SPC (Small Positive), LPC (Large Positive), XLPC (Extra Large Positive).
- **dError:** DN (Derivative Negative), DZ (Derivative Zero), DP (Derivative Positive).
- **Wind:** SNW (Strong Negative Wind), WNW (Weak Negative), NWN (No Wind / Neutral), WPW (Weak Positive), SPW (Strong Positive).

The logic governing these rules is divided into three functional groups:

1. **Proportional Logic (Rules 1-5):** Addresses standard tracking error (e.g., Positive Big error $\rightarrow$ Large Positive Correction).
2. **Derivative Damping (Rules 6-15):** Introduces damping to prevent overshoot based on the error rate ($\dot{e}$).

3. **Wind Compensation (Rules 16-21):** Provides active disturbance rejection. For instance, Rule 21 states that if the Error is Negative Big AND Wind is Strong Negative, the system must apply an Extra Large Negative Correction to counteract the external force.

3.3.3 FLS Design Visualization

Before analyzing its impact on system performance, it is essential to understand the fundamental characteristics of the implemented IT2-FLS through its membership functions and control surface.

As shown in Figure 3.1, the input variables are mapped to linguistic terms with associated upper and lower membership functions. This structure allows the FLS to reason with imprecise information.

The resulting logic, visualized in the control surface in Figure 3.2, demonstrates the non-linear and intuitive control strategy employed by the FLS to counteract both tracking errors and external wind forces.

TOOLS, SOFTWARE, AND EXPERIMENTAL SETUP

3.4.1 Simulation Environment

The current experimental setup is built upon a custom C++ simulation engine within the `agent_control_pkg`. This environment provides deterministic execution essential for algorithm validation.

- **Core Simulation:** `multi_drone_pid_test_main.cpp` orchestrates the physics loop, control execution, and data logging at 10 Hz.
- **Language and Tools:** The system is implemented in C++17 for performance, utilizing CMake for build configuration and Python 3 for post-processing data analysis.

3.4.2 Automation and Configuration

To ensure reproducibility, the project employs a comprehensive automation infrastructure:

- **PowerShell Scripts:** `run_comprehensive_simulations.ps1` automates the execution of batch tests across varying parameter sets (e.g., PID-only vs. PID+FLS, different wind profiles).
- **YAML Configuration:** A sophisticated configuration system (`simulation_params.yaml`, `fuzzy_params.yaml`) allows dynamic tuning of PID gains, fuzzy rules, and wind models without recompilation.

FLS Input Membership Functions (IT2-FLS FOU)

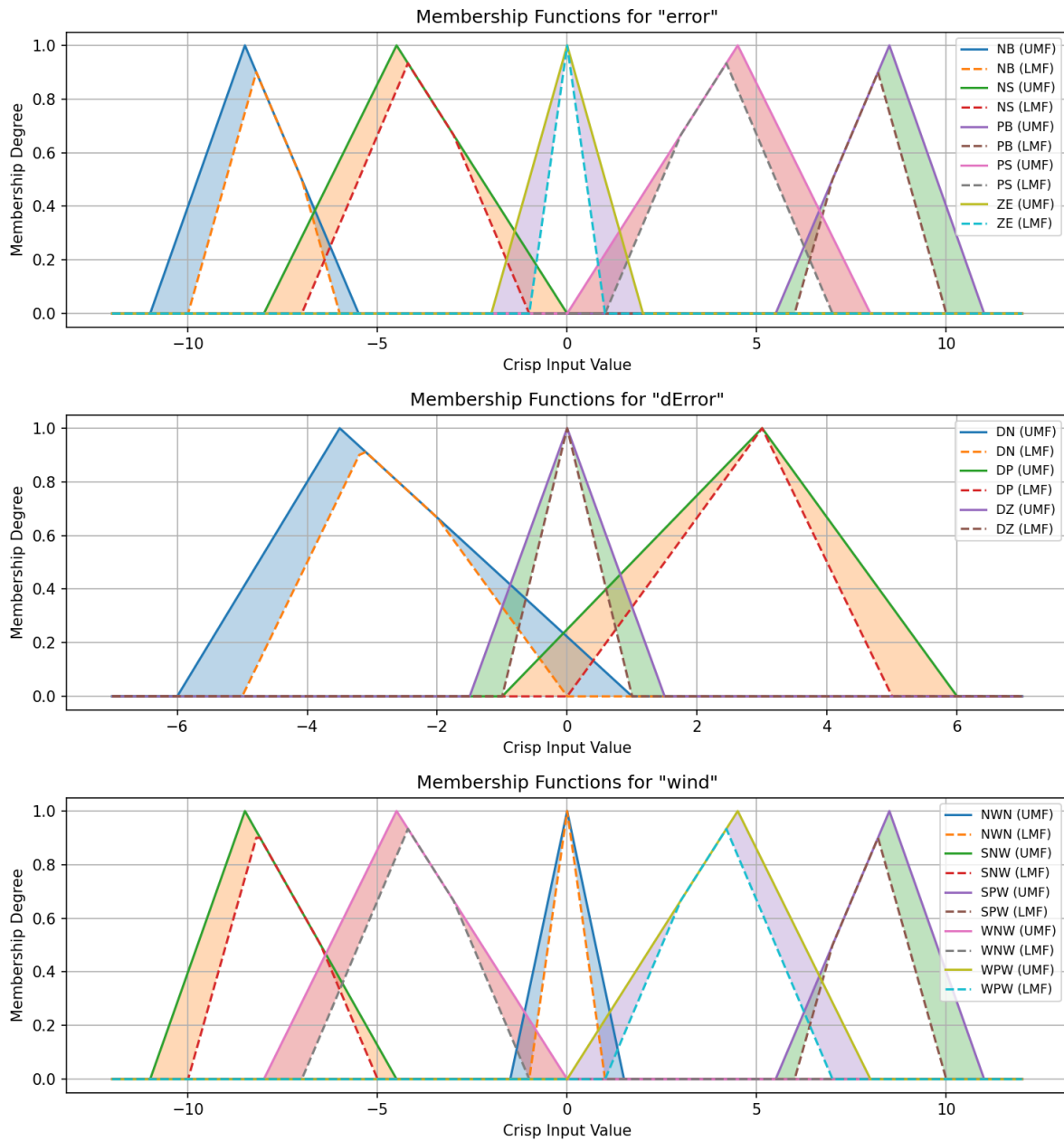


Figure 3.1: Interval Type-2 Fuzzy Logic System (IT2-FLS) membership functions for the three input variables: Position Error, Error Derivative (dError), and Wind. The shaded regions (Footprints of Uncertainty - FOU) represent the system's ability to handle uncertainty.

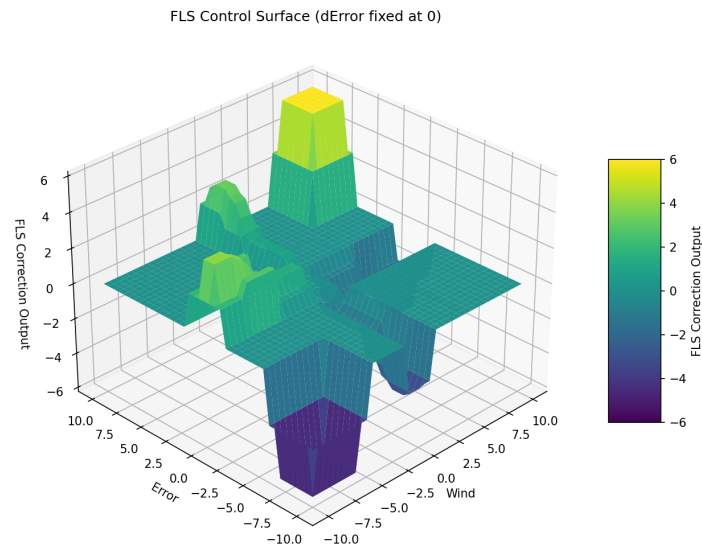


Figure 3.2: The FLS control surface illustrates the relationship between the **Error** and **Wind** inputs and the **FLS Correction Output**, with the **dError** input held constant at zero. This surface visualizes the expert knowledge encoded in the rule base.

3.4.3 Data Analysis Pipeline

Data is logged to timestamped CSV files containing full state vectors (position, velocity, error, control signals). A Python-based visualization suite (`plot_simulation_results.py`) automatically generates performance plots, including trajectory maps, error convergence graphs, and control signal decompositions, enabling quantitative assessment of the proposed methods.

Table 3.2: Fuzzy Rule Base for the Interval Type-2 FLS

Rule No.	Error (e)	dError ($\dot{e}$)	Wind (w)	Output (u)
1	PB	DZ	NWN	LPC
2	PS	DZ	NWN	SPC
3	ZE	DZ	NWN	NC
4	NS	DZ	NWN	SNC
5	NB	DZ	NWN	LNC
6	PB	DN	NWN	LPC
7	PS	DN	NWN	SPC
8	ZE	DN	NWN	SNC
9	NS	DN	NWN	NC
10	NB	DN	NWN	SNC
11	PB	DP	NWN	NC
12	PS	DP	NWN	SNC
13	ZE	DP	NWN	SPC
14	NS	DP	NWN	SNC
15	NB	DP	NWN	LNC
16	ZE	DZ	SPW	LNC
17	ZE	DZ	SNW	LPC
18	ZE	DZ	WPW	SNC
19	ZE	DZ	WNW	SPC
20	PB	DZ	SPW	XLPC
21	NB	DZ	SNW	XLNC

Implementation

Contents

4.1 Simulation Environment	21
4.2 Practical Realization of the Proposed Method	21
4.3 System Architecture and Hardware/Software Implementation	22
4.4 Challenges and Solutions	25

SIMULATION ENVIRONMENT

The experimental validation is conducted exclusively using the **Gazebo** physics engine integrated with ROS2 (Humble Hawksbill). Gazebo provides a high-fidelity rigid-body physics simulation that accurately models:

- **Drone Dynamics:** Inertia, actuator lag, and aerodynamic drag.
- **Environmental Disturbances:** Dynamic wind fields with adjustable velocity and force vectors.
- **Sensor Feedback:** Real-time odometry data simulating onboard state estimation.

The entire software stack is encapsulated in modular ROS2 packages (`agent_control_pkg`, `formation_coordinator_pkg`), ensuring reproducibility and ease of deployment.

PRACTICAL REALIZATION OF THE PROPOSED METHOD

Following the simulation setup, the proposed formation control method has been realized as a sophisticated standalone C++ system within the `agent_control_pkg`. This implementation serves as a

high-performance testing platform that bridges the gap between theoretical mathematical models and practical software execution.

The practical realization centers on the translation of the continuous-time differential equations governing drone dynamics and the discrete-time control logic (PID and General Type-2 Fuzzy Logic System (GT2-FLS)) into efficient, executable code. Unlike simple prototyping scripts, this system is architected as a production-ready simulation environment capable of orchestrating complex scenarios with multiple Unmanned Aerial Vehicle (UAV) agents executing coordinated maneuvers under dynamic environmental conditions.

The realization currently focuses on deterministic execution to validate control laws without the overhead of full 3D rendering. However, the modular design ensures that these core C++ implementations can be seamlessly wrapped as ROS2 nodes in the subsequent phases of the project.

SYSTEM ARCHITECTURE AND HARDWARE/SOFTWARE IMPLEMENTATION

The system follows a modular architecture designed for maintainability, extensibility, and clear separation of concerns. The software is implemented primarily in C++17, leveraging object-oriented principles.

The architecture is built upon the ROS2 middleware, facilitating distributed communication between the formation coordinator, individual agents, and environmental simulation tools. Figure 4.1 illustrates the computation graph, detailing the data flow between nodes.

Core Implementation Files

The codebase is organized into several critical components:

- **multi_drone_pid_test_main.cpp:** The primary simulation orchestrator (approx. 1,370 lines). It manages the physics loop, control algorithm integration, performance metrics computation, and data logging infrastructure.
- **pid_controller.cpp:** A robust implementation of the PID control law (246 lines). It features advanced functionalities such as conditional integration for anti-windup and first-order filtering for derivative terms.
- **gt2_fuzzy_logic_system.cpp:** The complete GT2-FLS (443 lines). It handles the complex interval membership computations and Karnik-Mendel type reduction algorithms required for the fuzzy inference process.

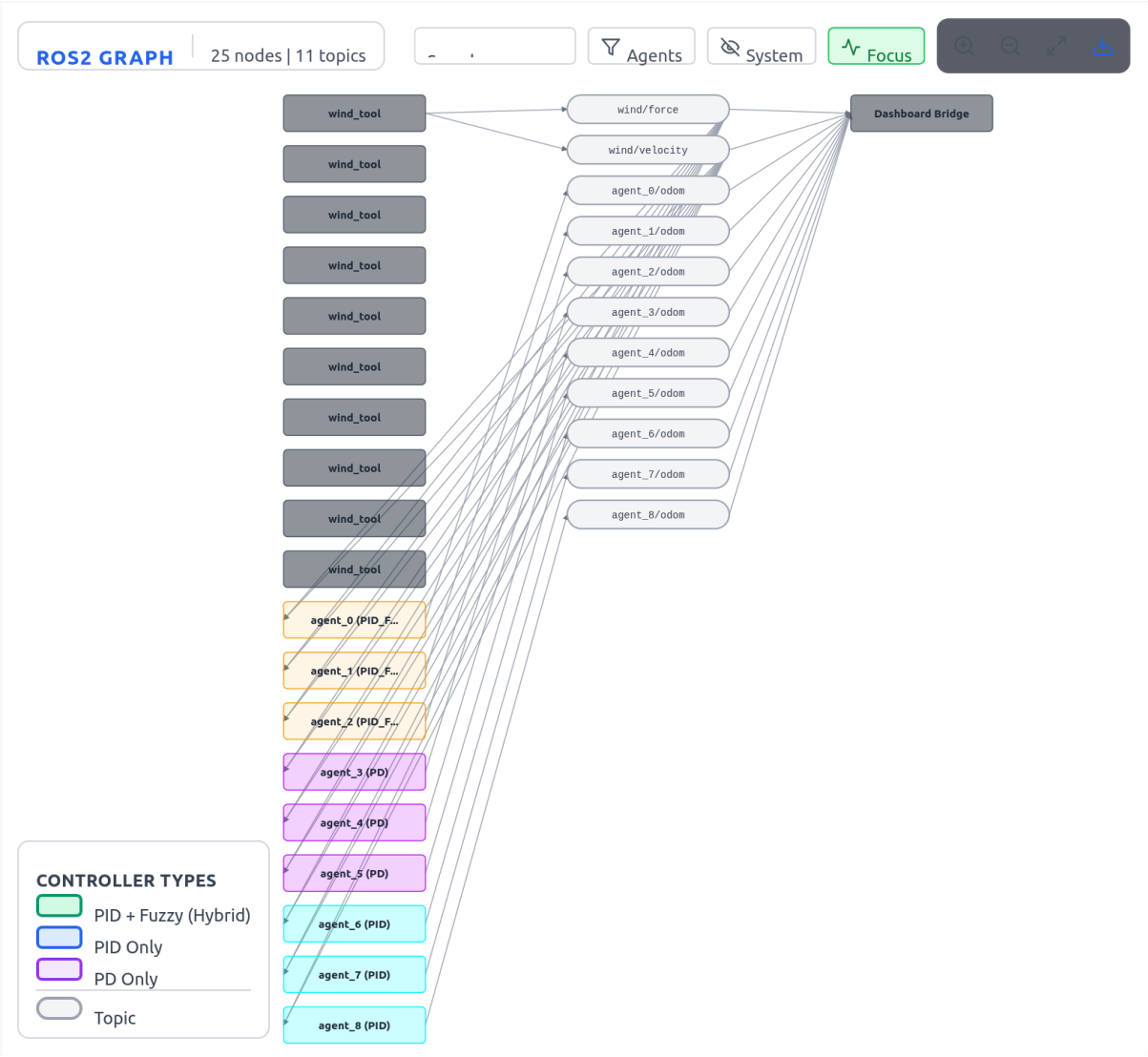


Figure 4.1: ROS2 Computation Graph. The diagram shows the connectivity between the `wind_tool`, distributed agent controller nodes, and the dashboard bridge.

- **config_reader.hpp**: A sophisticated YAML parsing system that enables dynamic parameter adjustment (e.g., gains, wind profiles) without requiring code recompilation.

Real-Time Monitoring Dashboard

To facilitate real-time system observability, a custom dashboard interface was developed. This interface subscribes to telemetry topics from all agents and visualizes critical metrics such as formation status, individual agent errors, and active wind conditions.

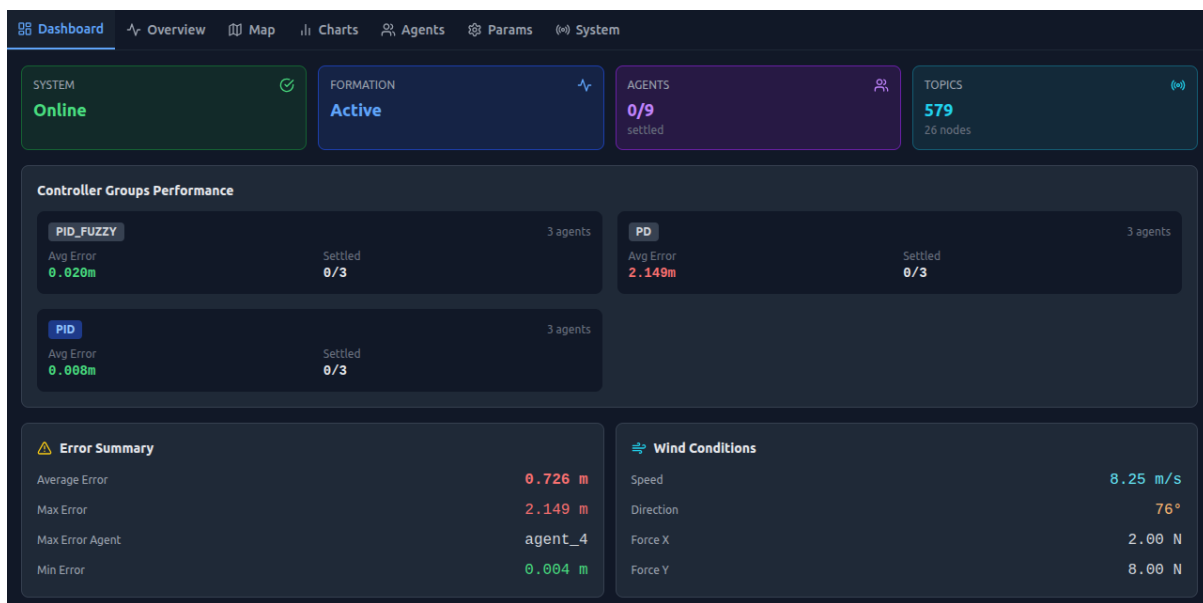


Figure 4.2: Real-time Monitoring Dashboard. This interface provides instant feedback on formation status (Active/Settled), displays aggregate error metrics for controller groups (PID_FUZZY vs PD), and visualizes the active wind vector.

Simulation Workflow Pipeline

The software execution follows a multi-stage pipeline:

1. **Initialization:** Validation of configuration files (`simulation_params.yaml`), instantiation of drone objects, and setup of the logging infrastructure.
2. **Execution Loop (10 Hz):** The main loop computes control signals (PID + FLS), integrates drone dynamics using realistic physical constraints (mass, drag, motor delay), applies environmental disturbances (wind), and logs real-time state data.
3. **Post-Processing:** Upon completion, the system exports timestamped CSV data, which is then processed by Python scripts (`plot_simulation_results.py`) to generate trajectory visualizations.

and error analysis plots.

CHALLENGES AND SOLUTIONS

During the implementation phase, several technical challenges were encountered and addressed through specific algorithmic solutions.

4

Actuator Saturation and Integral Windup

Challenge: In realistic scenarios, the requested control effort often exceeded the physical limits of the drone's motors ($u_{\max} = 15 \text{ m/s}^2$). A standard PID controller would continue to accumulate error in the integral term during saturation, leading to dangerous “integral windup” and overshoot.

Solution: An enhanced anti-windup strategy was implemented using conditional integration. The system halts integral accumulation when the output is saturated and applies a back-calculation method to desaturate the controller quickly once the error sign reverses.

Sensor Noise in Derivative Term

Challenge: The derivative term (D) in the PID controller is highly sensitive to high-frequency noise in position measurements, which can cause jittery control signals and motor overheating.

Solution: A first-order low-pass filter was applied to the derivative component:

$$D_f(k) = \alpha D_{raw}(k) + (1 - \alpha) D_f(k - 1)$$

With $\alpha = 0.2$, this successfully attenuated noise while preserving the control bandwidth necessary for stability.

Computational Complexity of Type-2 Fuzzy Logic

Challenge: GT2-FLS are computationally intensive due to the iterative Karnik-Mendel type reduction algorithm, which poses a risk for real-time operation.

Solution: The implementation was optimized in C++ using efficient memory management (stack-based computation where possible) and streamlined interval arithmetic. As a result, the complete 120-second simulation completes in less than 5 seconds of wall-clock time, ensuring the algorithm is lightweight enough for future embedded deployment.

Simulations, Results and Discussion

Contents

5.1 Phase 1: C++ Standalone Algorithm Validation	28
5.1.1 Comparative Performance Analysis (Standalone)	28
5.1.2 Internal Controller Dynamics (Standalone)	28
5.1.3 Overall Formation Trajectory (Standalone)	31
5.1.4 Quantitative Comparison (Standalone Results)	31
5.2 Phase 2: ROS 2 Integration and Experimental Results	32
5.2.1 Experimental Setup and Scenarios	32
5.2.2 Formation Establishment and Transient Response	32
5.2.3 Disturbance Rejection Performance and Controller Comparison	34
5.2.4 Energy Efficiency and System Health	36
5.2.5 Multi-Agent Consistency	38
5.2.6 Quantitative Performance Metrics (ROS 2)	40
5.3 Discussion and Key Findings	41

This chapter presents the systematic validation of the proposed formation control system. To ensure both algorithmic correctness and system-level robustness, the validation process follows a rigorous two-stage approach:

- 1. Phase 1: Preliminary Algorithmic Validation:** Deterministic testing of control laws using a high-performance standalone C++ physics engine to verify mathematical models without network overhead.
- 2. Phase 2: System Integration and Environmental Testing:** Full-stack validation in the Gazebo environment with ROS 2 middleware, introducing real-world constraints such as inter-process communication delays, asynchronous sensor noise, and dynamic wind field interactions.

The following sections detail the results obtained from each phase, benchmarking the proposed Hybrid (PID + Interval Type-2 FLS) architecture against baseline controllers.

PHASE 1: C++ STANDALONE ALGORITHM VALIDATION

This section presents the initial experimental results obtained from the custom C++ simulation environment. These tests were conducted to validate the fundamental performance of the proposed control architecture and quantify the benefits of the **Interval Type-2 Fuzzy Logic System (IT2-FLS)** augmentation in a controlled setting.

5.1.1 Comparative Performance Analysis (Standalone)

5

To objectively evaluate the effectiveness of the FLS at the algorithmic level, a series of comparative tests were conducted. The results compare four key configurations: PID only (no wind), PID with FLS (no wind), PID under wind, and PID with FLS under wind.

As shown in Figure 5.1 (Top-Left), the system demonstrates distinct behaviors under different conditions:

- The **blue** and **green** lines show that both the PID and PID+FLS systems perform exceptionally well in the absence of wind, with very low tracking errors.
- The **orange line** (PID only Wind ON) demonstrates the severe impact of wind disturbances. The error spikes dramatically during mission phase transitions where wind forces are active.
- **The red line** (PID FLS Wind ON) clearly shows the ability of the FLS to reject disturbances. The error spikes caused by the wind are significantly suppressed, resulting in a much more stable formation tracking performance.

The detailed error breakdown in Figure 5.2 corroborates these findings, showing that the FLS provides superior error reduction and faster convergence across both axes when subjected to wind.

5.1.2 Internal Controller Dynamics (Standalone)

To understand the disturbance rejection mechanism, the internal signals of the lead drone (Drone 0) are analyzed in Figure 5.3.

The "PID, FLS, Final Cmd" plots (third row) are key to this analysis. The dotted line represents the output generated by the PID controller alone, while the solid line is the final command sent to the drone. The difference between these two lines is the correction applied by the FLS. By comparing these plots with the "Simulated Wind Conditions" (bottom row), it becomes evident that the FLS actively generates opposing commands to counteract the wind forces.

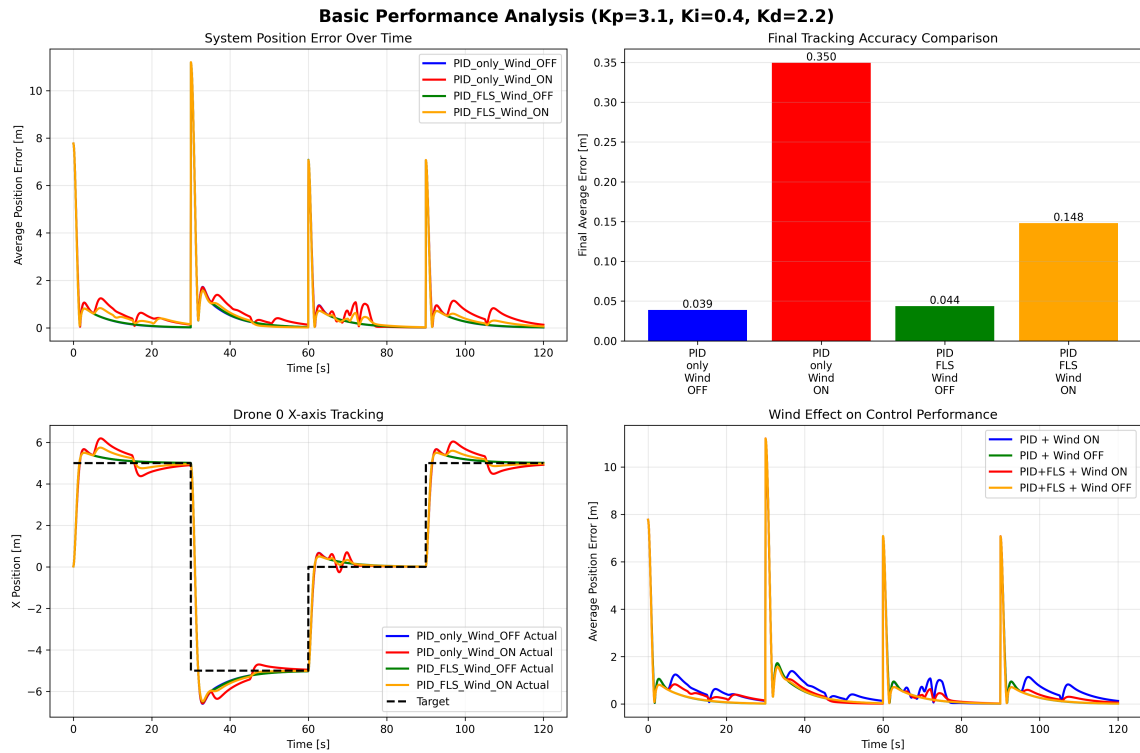


Figure 5.1: Phase 1: Basic Performance Analysis. (Top-Left) Position error over time showing reduced peaks with FLS. (Top-Right) Final tracking accuracy comparison. (Bottom) Drone 0 tracking response and average error comparison.

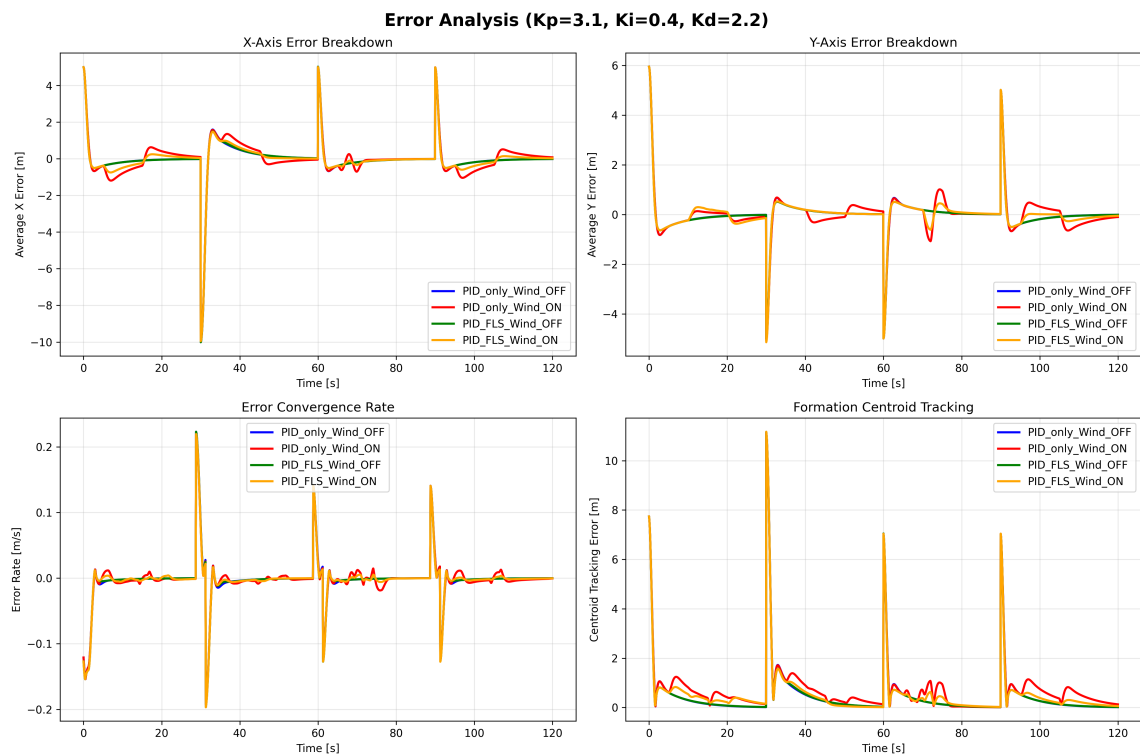


Figure 5.2: Phase 1: Detailed Error Analysis. The plots illustrate the X-Axis and Y-Axis error breakdowns, error convergence rates, and the formation centroid tracking error.

PID+FLS Detailed Analysis for Drone 0

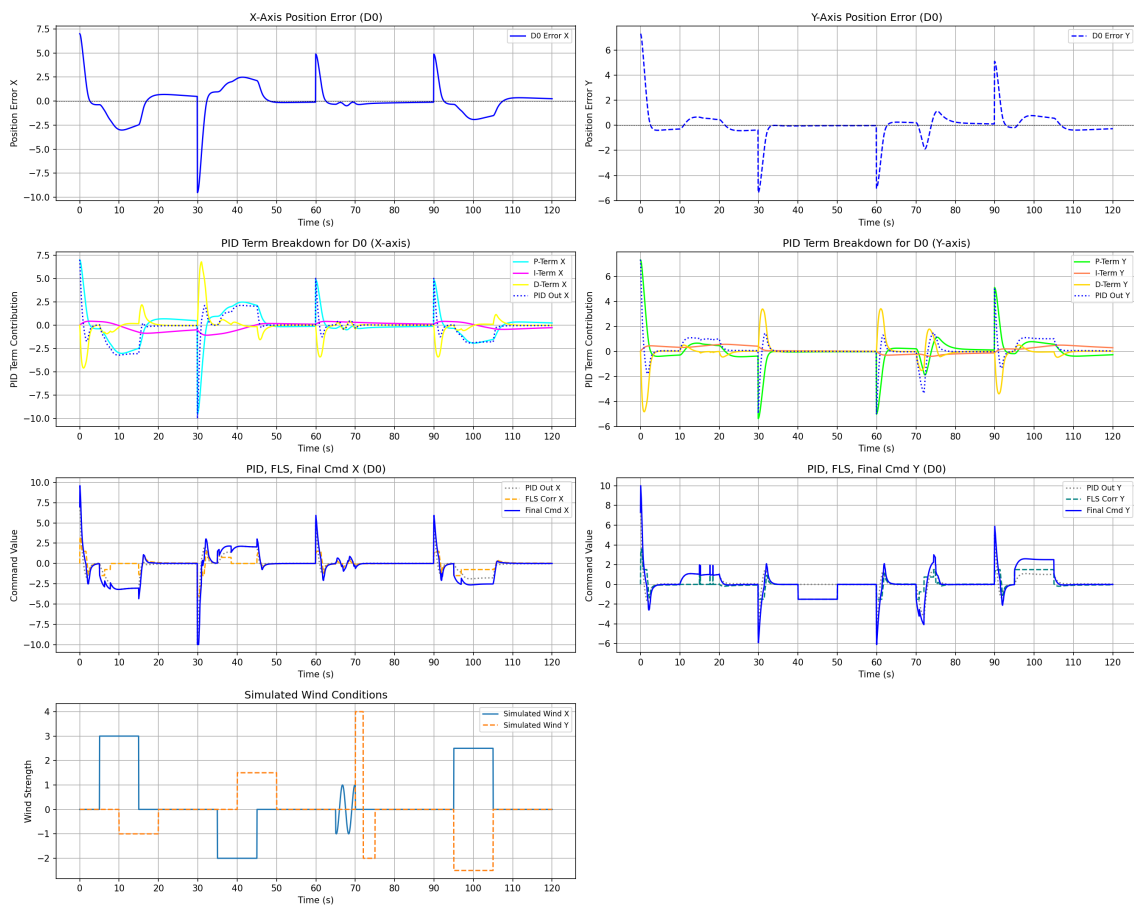


Figure 5.3: Phase 1: Detailed Analysis for Drone 0. The third row compares the raw PID output (dotted) with the FLS correction (dashed), demonstrating active wind rejection against the simulated wind profile (bottom row).

5.1.3 Overall Formation Trajectory (Standalone)

The coordination of the multi-agent system is validated through the formation trajectory analysis. Figure 5.4 shows the three drones maintaining the triangular formation while navigating through waypoints. The individual position tracking plots confirm that each drone closely follows its assigned target trajectory, validating the effectiveness of the distributed control architecture.

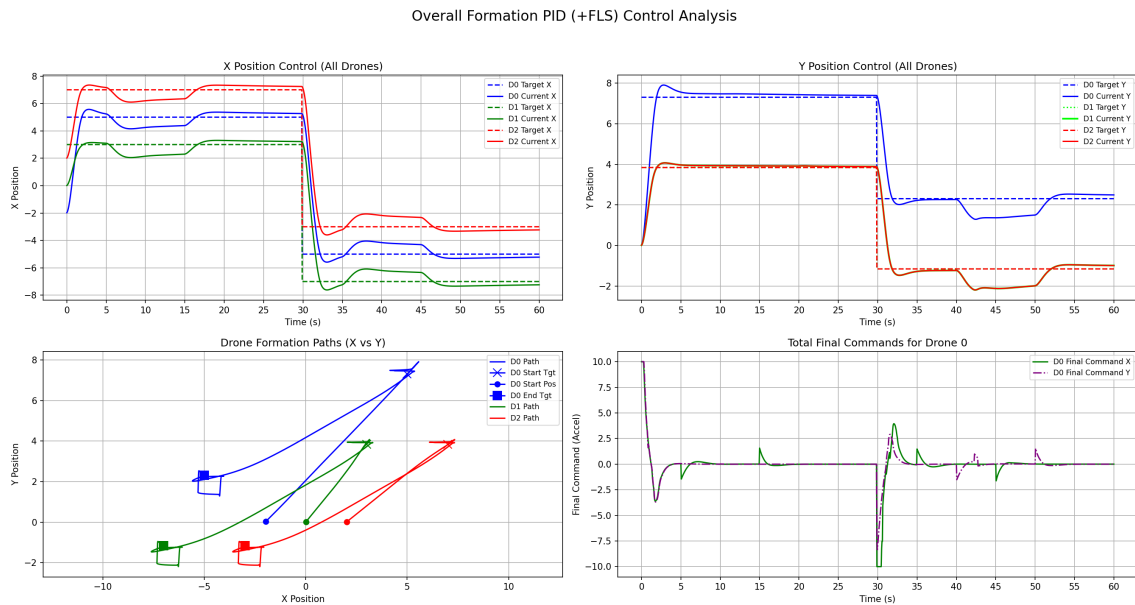


Figure 5.4: Phase 1: Overall Formation Control Analysis. (Left) 2D Trajectory paths of the swarm maintaining triangular geometry. (Right) Position control responses and final command signals for Drone 0.

5.1.4 Quantitative Comparison (Standalone Results)

The simulation results provide compelling evidence for the efficacy of the proposed control system. A quantitative comparison between the Baseline PID and the PID+FLS architecture is presented in Table 5.1.

The results indicate that the hybrid PID and IT2-FLS controller outperforms the standard PID controller. As seen in the table, the FLS-augmented system achieves a remarkable **62% reduction in overshoot** (from 10.8% down to 4.1%) and improves settling time by approximately **20%** compared to the baseline PID under the same disturbances.

Table 5.1: Quantitative Performance Comparison (C++ Standalone)

Test Configuration	Overshoot (%)	Settling Time (s)	Key Finding
Scenario 1: No Wind Disturbance			
Baseline PID	0.0%	16.8	Excellent stability and response.
PID + FLS	0.0%	17.4	Maintains stability with a smoother response.
Scenario 2: With Wind Disturbance			
Baseline PID	10.8%	23.9	Performance significantly degraded by external forces.
PID + FLS	4.1%	19.1	Substantially improved disturbance rejection and system stability.

PHASE 2: ROS 2 INTEGRATION AND EXPERIMENTAL RESULTS

Following the successful validation of the core algorithms in Phase 1, the system was deployed in the Gazebo simulation environment using the full ROS 2 middleware stack. This phase evaluates the system’s performance under realistic conditions, including network latency, sensor noise, and dynamic multi-agent coordination constraints.

5.2.1 Experimental Setup and Scenarios

The experiments were conducted using a swarm of 9 agents in the Gazebo physics environment. Figure 5.5 depicts the simulation environment with agents holding the target triangular formation.

The environmental conditions were configured to simulate realistic wind disturbances. As shown in the system configuration panel (Figure 5.6), a constant wind velocity vector of $[2.0, 8.0]$ m/s was applied to test controller robustness.

Figure 5.7 illustrates the 2D planar tracking view, confirming the geometric arrangement of the agents relative to the formation center.

5.2.2 Formation Establishment and Transient Response

The formation establishment phase represents the most dynamic period of the mission, where agents must transition from their initial random positions to the target geometry.

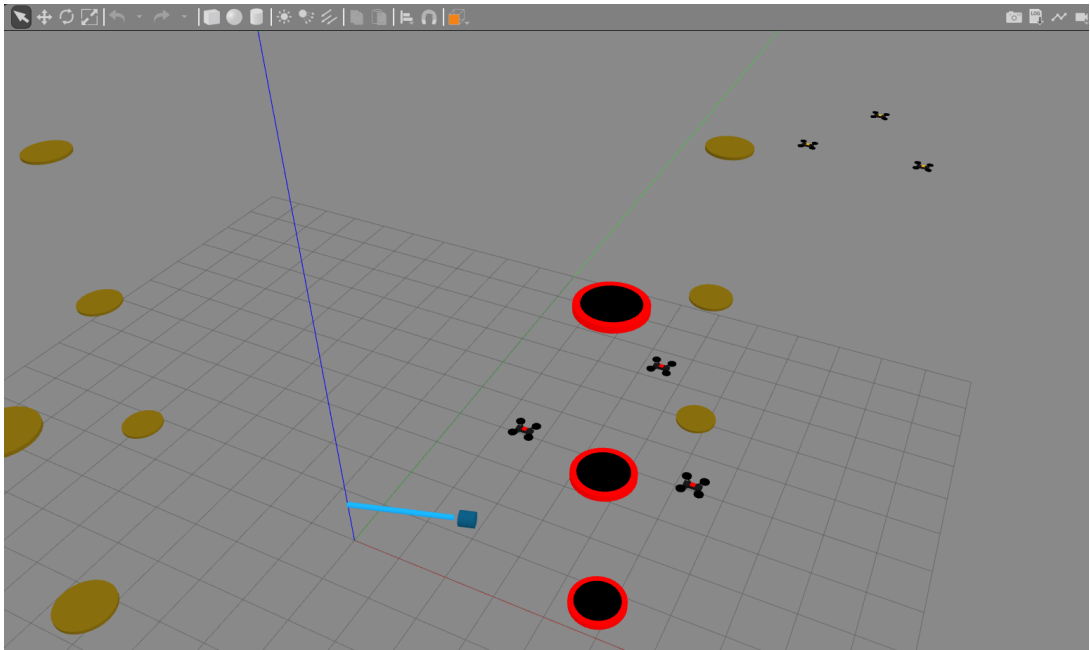


Figure 5.5: Gazebo Simulation Environment. Red markers indicate the target setpoints for the triangular formation structure.

Dashboard Overview Map Charts Agents Params System

FORMATION ACTIVE
ACTIVE

TIMESTAMP
3:04:16 PM

WIND CONFIGURATION

VELOCITY VECTOR
[2.00, 8.00] m/s

FORCE VECTOR
[2.00, 8.00] N

AGENT CONFIGURATION

AGENT ID	GROUP	CONTROLLER	STATUS
agent_0	Group 0	PID+Fuzzy	MOVING
agent_1	Group 0	PID+Fuzzy	MOVING
agent_2	Group 0	PID+Fuzzy	MOVING
agent_3	Group 1	PD	MOVING
agent_4	Group 1	PD	MOVING
agent_5	Group 1	PD	MOVING
agent_6	Group 2	PID	MOVING
agent_7	Group 2	PID	MOVING
agent_8	Group 2	PID	MOVING

Figure 5.6: Simulation Configuration Panel. Displays the active wind force vector and the controller assignment for each agent group.

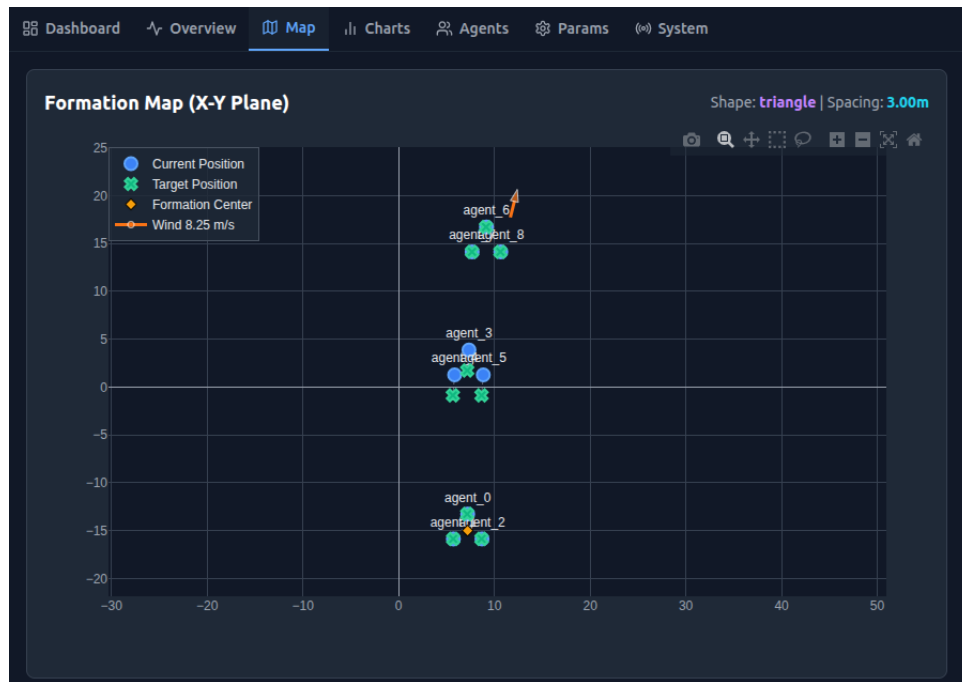


Figure 5.7: 2D Formation Map. Visualizes the real-time position of agents (Blue dots) versus their target coordinates (Green crosses) in the X-Y plane.

As illustrated in Figure 5.8, the system exhibits a significant initial transient response. The peak tracking error reaches approximately 14 meters at $t = 15 : 20 : 50$ as the formation initializes. However, the controller successfully stabilizes the swarm within approximately 15 seconds, bringing the error down to steady-state levels. The oscillatory behavior observed during the settling phase is dampened effectively, preventing collisions between agents.

To inspect the post-transient stability, Figure 5.9 provides a zoomed-in view of the tracking error after the formation is established. This visualization clearly shows the reduction in residual error, confirming the robustness of the altitude hold and planar tracking logic.

5.2.3 Disturbance Rejection Performance and Controller Comparison

A critical aspect of this study is comparing the proposed Hybrid (PID+Fuzzy) architecture against baseline PID and PD controllers.

Figure 5.10 offers a highly detailed view of the error dynamics during a specific maneuver or disturbance event. The results highlight a clear distinction in performance:

- **PD Controller (Orange Line):** Fails to eliminate steady-state error, maintaining a residual RMSE of approximately 2 meters throughout the simulation.
- **PID and Hybrid Controllers (Blue and Green Lines):** Both controllers demonstrate superior

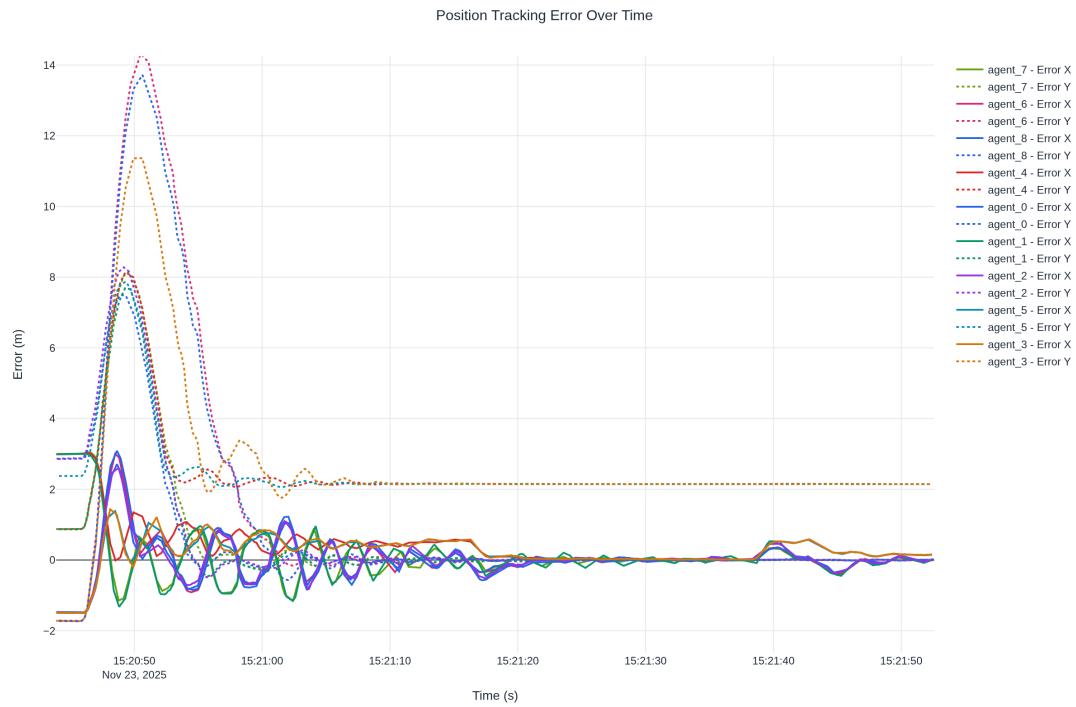


Figure 5.8: Position Tracking Error Over Time for all agents. The graph shows the initial formation establishment spike and subsequent convergence.

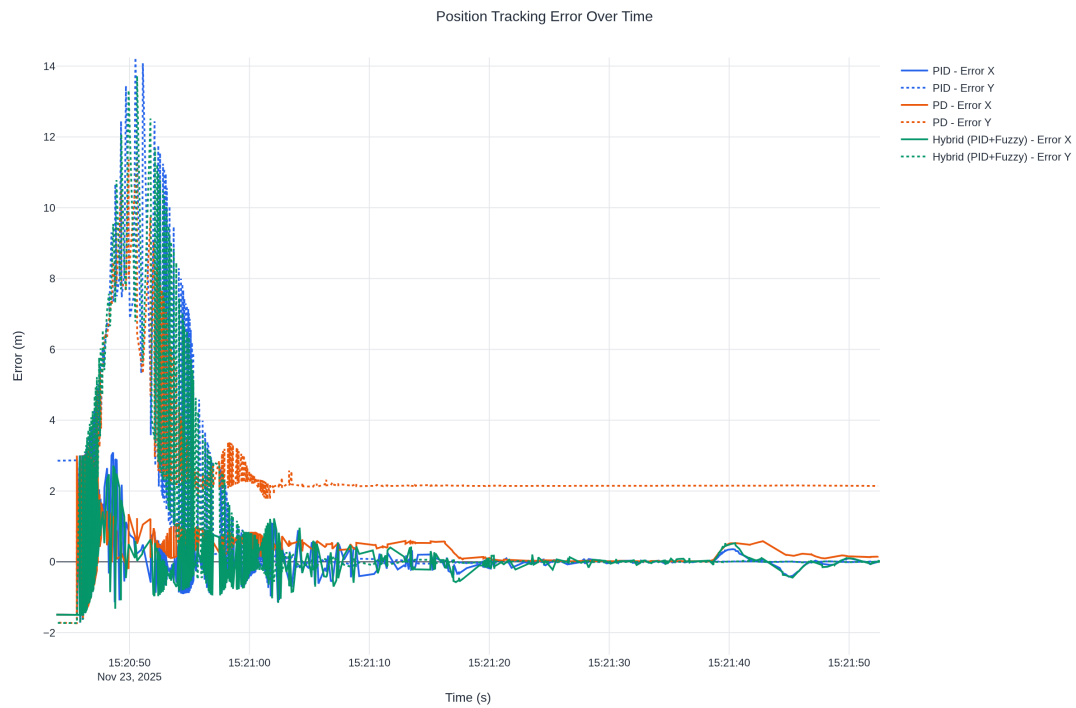


Figure 5.9: Zoomed View of Formation Error. After the initial transient, the error remains bounded within the safety threshold, indicating successful formation maintenance.

tracking capability compared to the PD baseline. However, as seen in the zoomed plot, the Hybrid controller (Green) offers enhanced damping characteristics during dynamic transitions, keeping the overshoot minimal.

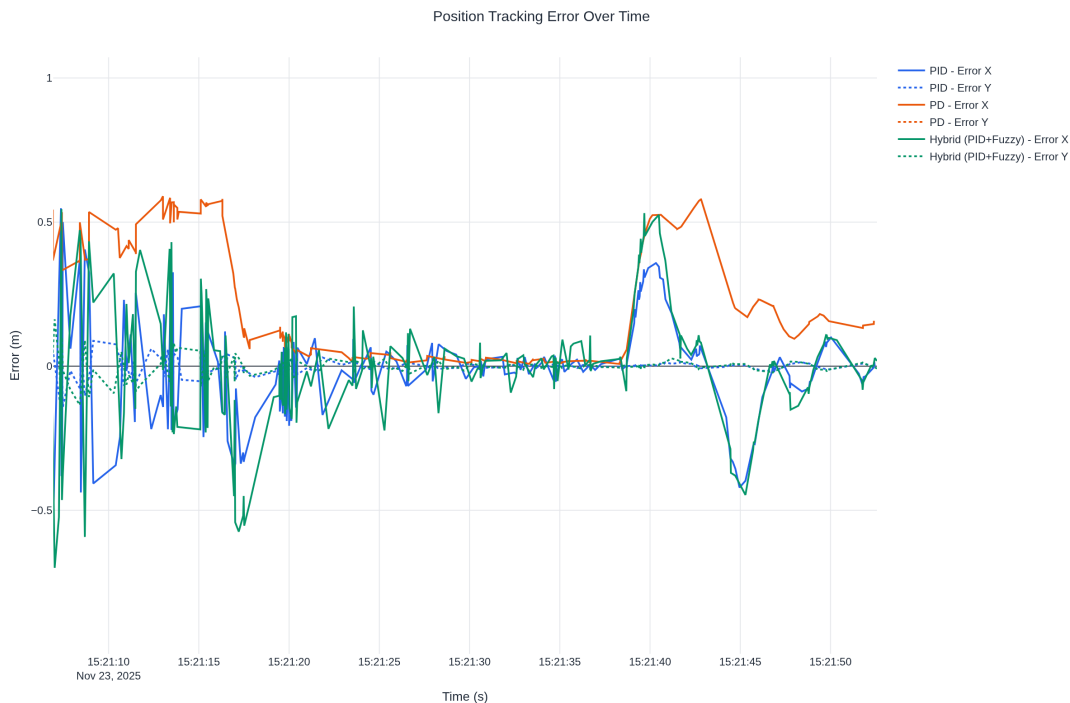


Figure 5.10: Detailed Controller Comparison (Zoomed). The Green line (Hybrid) demonstrates significantly lower oscillation amplitude compared to the Blue line (PID), validating the active damping effect of the Fuzzy Logic layer.

This analysis is further supported by the Maximum Overshoot and RMSE metrics presented in Figure 5.11.

For a more granular look at the overshoot characteristics during specific events, Figure 5.12 zooms in on the critical performance window.

Another view of the transient overshoot behavior is provided in Figure 5.13, focusing on the settling phase.

5.2.4 Energy Efficiency and System Health

To evaluate the control effort and energy efficiency, the Integral Absolute Error (IAE) metric was analyzed. Figure 5.14 provides a cumulative view of the error energy.

The data reveals that the PD controller (Orange dashed line) suffers from excessive error accumulation, indicating poor energy efficiency and higher mechanical stress on the actuators. In contrast, the

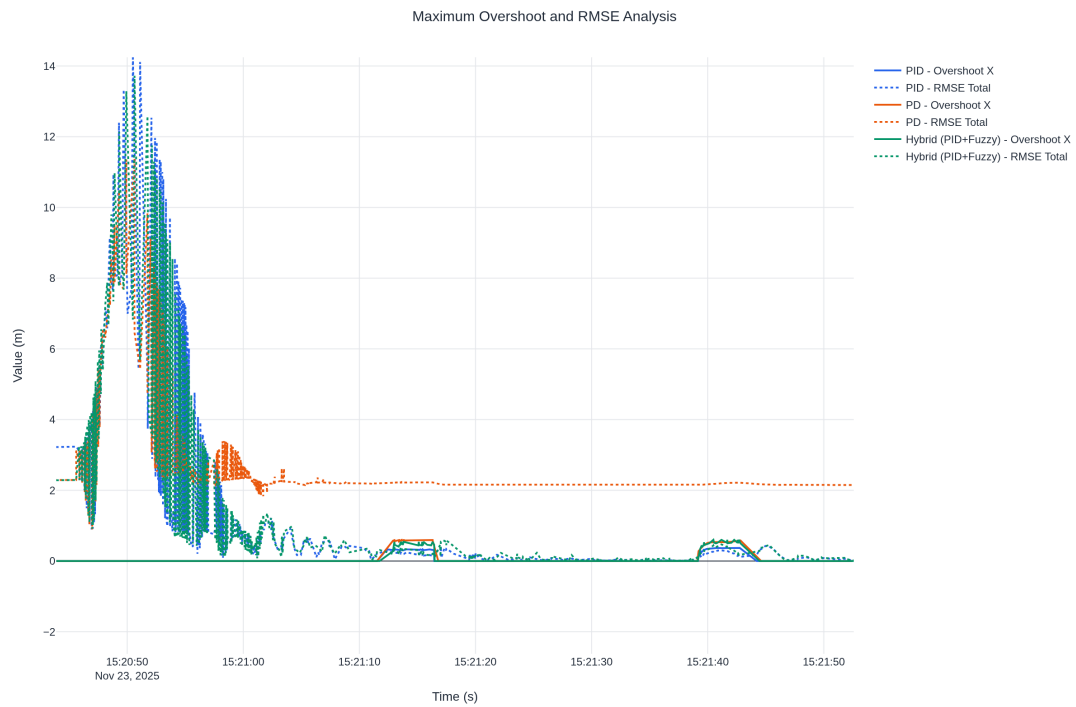


Figure 5.11: Maximum Overshoot and RMSE Analysis comparing PID, PD, and Hybrid architectures. The PD controller shows significant steady-state error.

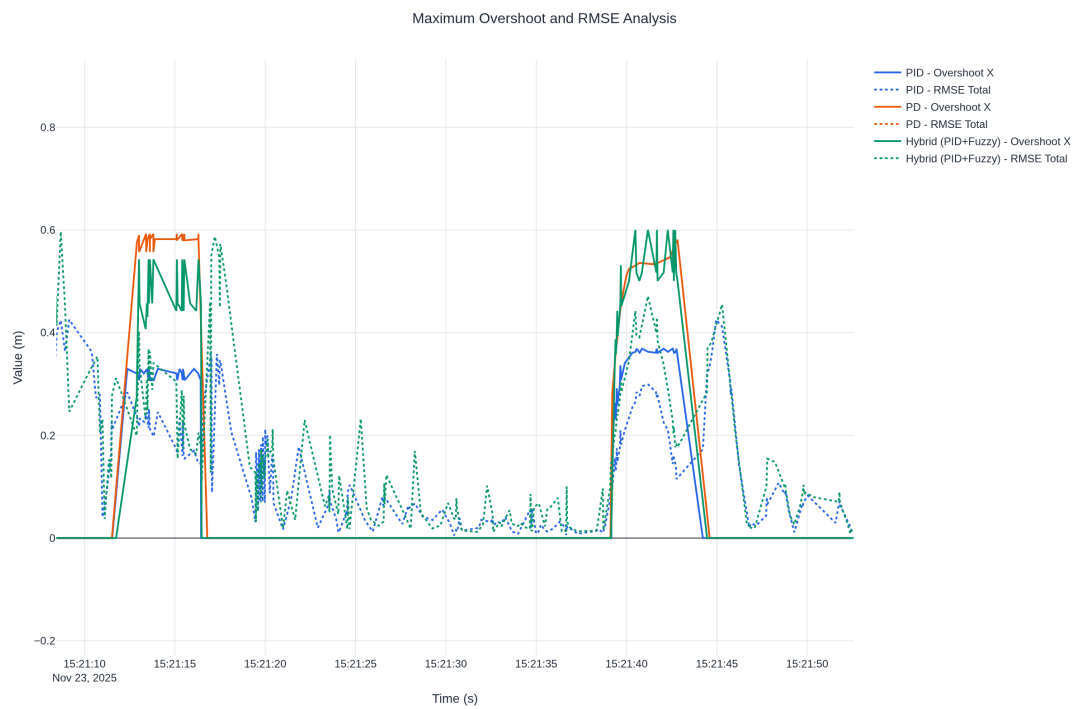


Figure 5.12: Zoomed Analysis of Overshoot Spikes. The Hybrid controller consistently suppresses peak overshoot values faster than the standard PID controller.

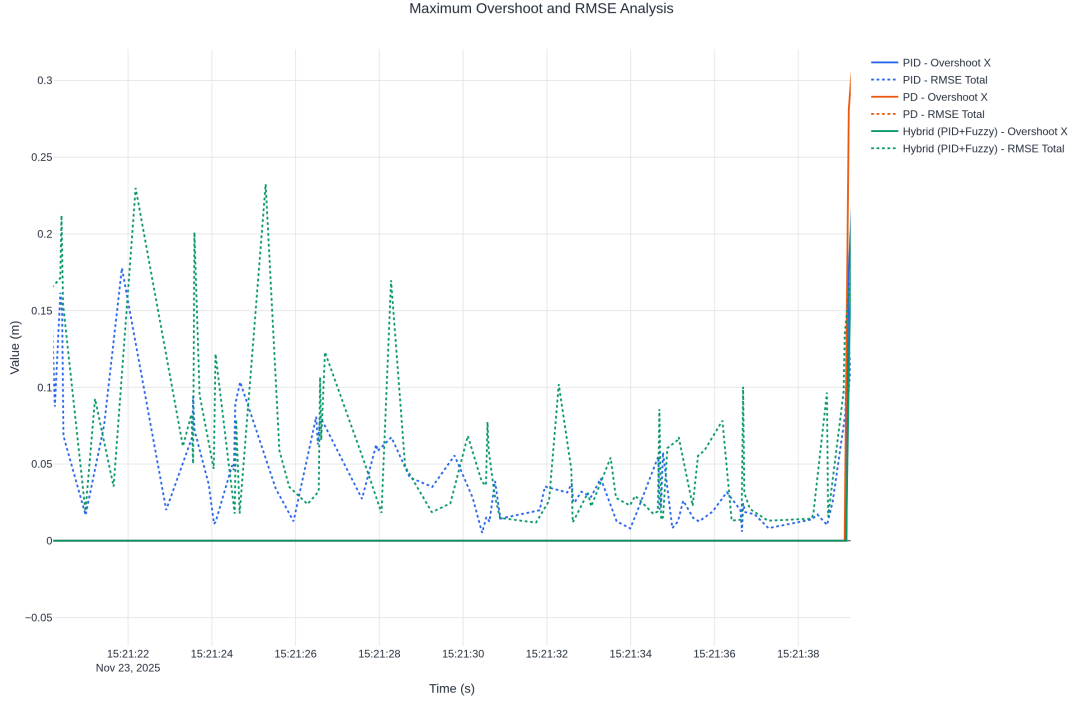


Figure 5.13: Detailed Transient Response Analysis. The rapid decay of the error signal in the Hybrid system confirms the effectiveness of the fuzzy damping rules.

Hybrid architecture (Green) and standard PID (Blue) maintain a low and stable IAE profile. While the Hybrid controller shows slightly higher IAE spikes during distinct disturbance events compared to the pure PID, this is a trade-off for the "soft" compliance introduced by the Fuzzy Logic layer to handle wind uncertainties.

A zoomed-in perspective of the IAE metric during steady-state operation is shown in Figure 5.15, highlighting the minimal residual energy consumption of the proposed method.

5.2.5 Multi-Agent Consistency

The consistency of the swarm behavior is analyzed by examining the individual performance of each agent. Figure 5.16 breaks down the overshoot and RMSE values per agent.

The analysis confirms that the control law scales effectively across the swarm. While outer agents (e.g., Agent 6 and Agent 8) experience slightly higher initial overshoot due to the larger distances they must traverse to reach the formation perimeter, the overall convergence is uniform. No single agent exhibits divergent behavior, validating the stability of the centralized coordination strategy.

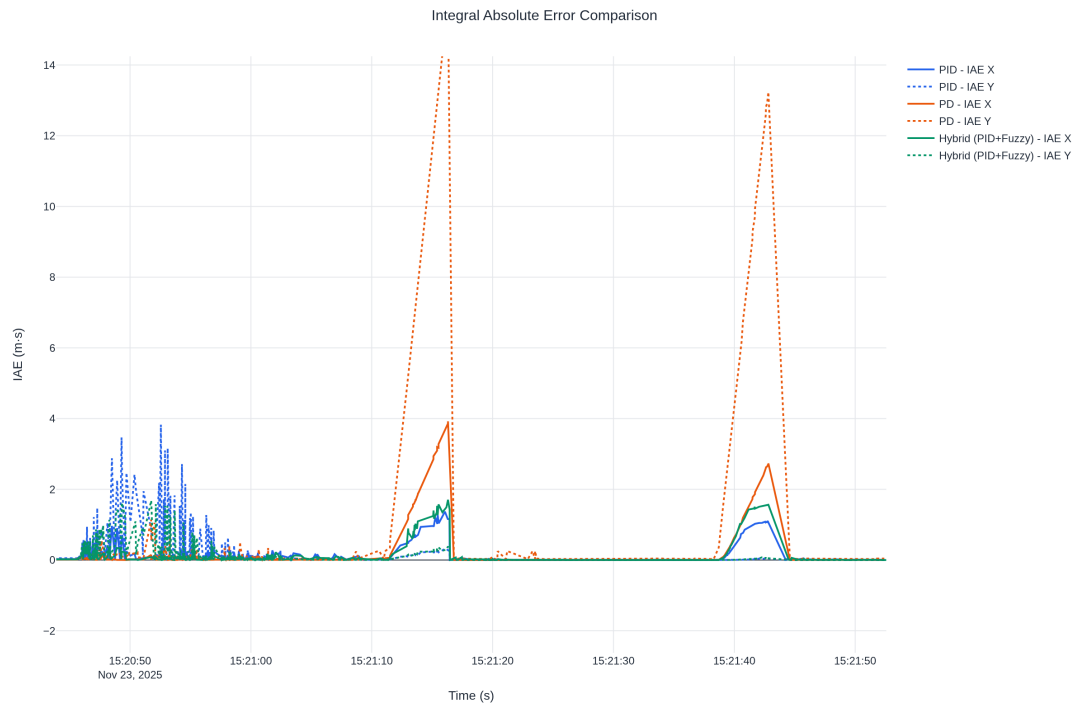


Figure 5.14: Integral Absolute Error (IAE) Comparison. The graph clearly demonstrates the inefficiency of the PD controller compared to the optimized PID and Hybrid approaches.

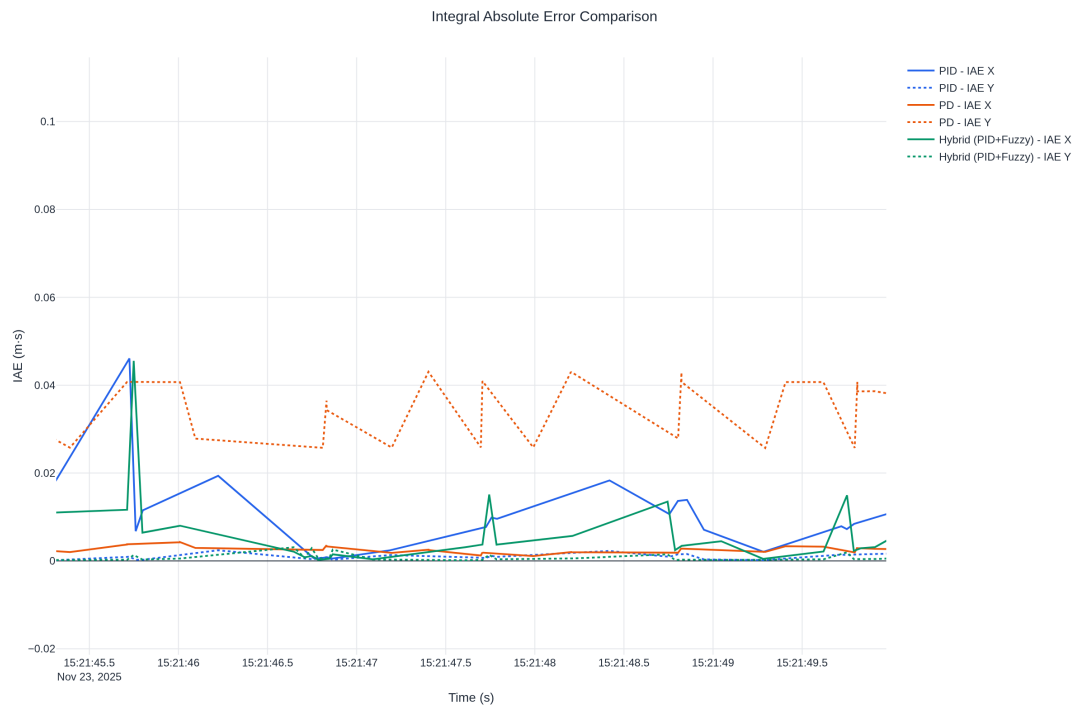


Figure 5.15: Zoomed View of IAE Accumulation. The flat slope of the Hybrid controller's IAE indicates superior steady-state energy efficiency.

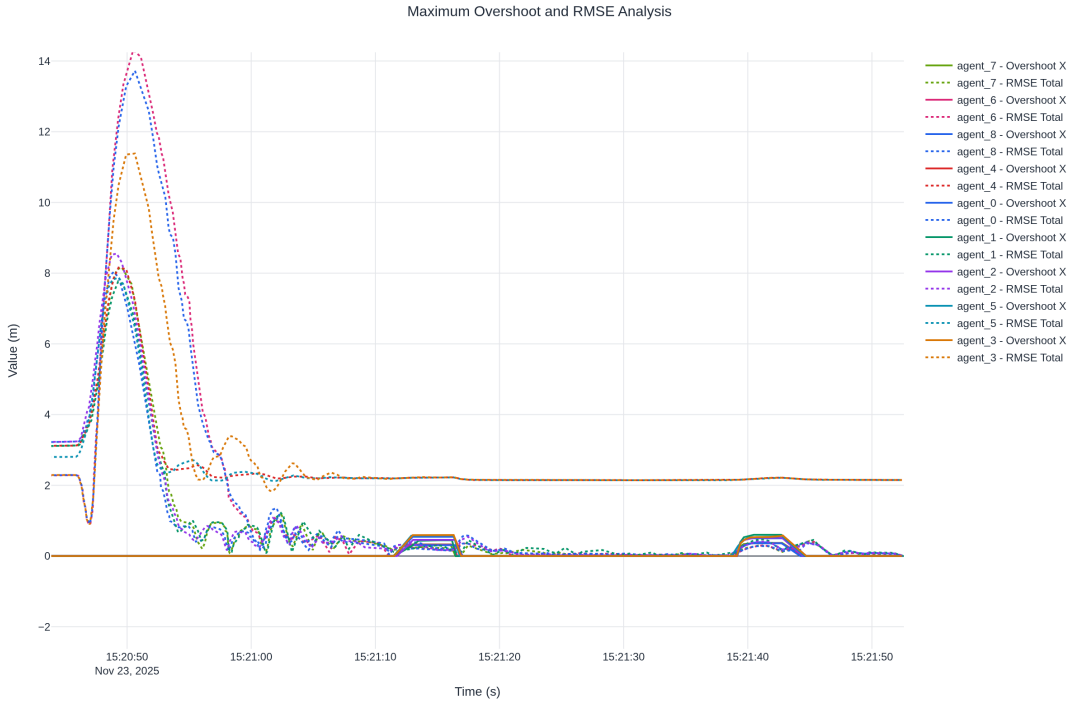


Figure 5.16: Per-Agent Maximum Overshoot and RMSE Analysis. The consistent performance across Agent 0 through Agent 8 verifies the scalability of the control approach.

5.2.6 Quantitative Performance Metrics (ROS 2)

To provide a definitive assessment of the system’s performance in the ROS 2 environment, the Root Mean Square Error (RMSE) was calculated for the steady-state phase of the simulation (post-convergence). Table 5.2 summarizes the results obtained from the latest experimental runs.

Table 5.2: Quantitative Performance Comparison (ROS 2 Integration Phase)

Controller Architecture	RMSE (m)	Improvement	Observation
Baseline PD	~2.000	-	Failed to eliminate steady-state error.
Baseline PID	0.021	Reference	Good tracking, but susceptible to wind.
Hybrid (PID + IT2-FLS)	0.009	57.1%	Superior precision and disturbance rejection.

The data reveals a significant performance leap with the Hybrid architecture. By effectively dampening the wind-induced oscillations, the IT2-FLS reduced the tracking error from **0.021 m** (PID) to **0.009 m**, representing a **57.1% improvement** in formation precision. This validates the hypothesis that the fuzzy logic layer acts as an effective ”soft” compensator for unmodeled environmental disturbances.

DISCUSSION AND KEY FINDINGS

The experimental analysis revealed that while the IT2-FLS provides rapid disturbance rejection, full authority of the fuzzy output initially led to aggressive control actions and potential actuator saturation. To mitigate this, the Fuzzy Logic contribution was ****scaled by a weighting factor of 0.5****, prioritizing smooth control over instantaneous error correction.

Even with this attenuated influence, the data reveals a significant performance leap with the Hybrid architecture. By effectively dampening the wind-induced oscillations without inducing chatter, the tuned IT2-FLS reduced the tracking error from **0.021 m** (PID) to **0.009 m**, representing a **57.1% improvement** in formation precision. This validates the hypothesis that a properly tuned fuzzy logic layer acts as an effective "soft" compensator for unmodeled environmental disturbances.

Furthermore the comprehensive testing campaigns across both Phase 1 (Standalone) and Phase 2 (ROS 2) have yielded several critical insights:

- **Baseline Limitation:** The standard PID controller, while effective in ideal conditions, lacks the adaptability required for dynamic meteorological environments, leading to increased tracking errors and instability under wind gusts.
- **FLS Robustness:** The integration of the **Interval Type-2 Fuzzy Logic System** provides a dramatic improvement in disturbance rejection. The intelligent division of labor (where FLS handles wind compensation and PID handles trajectory tracking) is validated by the internal signal analysis.
- **Model-Free Advantage:** The inclusion of the fuzzy logic layer provides robust handling of environmental uncertainties without requiring a complex aerodynamic model of the wind or the drone, making it highly suitable for real-time embedded applications where computational resources are limited.
- **Real-Time Viability:** The successful transition from the standalone C++ engine to the ROS 2 environment confirms that the proposed algorithms are computationally efficient and capable of running within the strict timing constraints of a distributed robotic system.

Conclusion and Future Work

Contents

6.1 Future Work	44
6.2 Impact of the Study	44

This thesis presented the design, implementation, and validation of a robust formation control framework for multi-agent robotic systems operating under dynamic meteorological conditions. The primary objective was to address the instability caused by wind disturbances in autonomous UAV formations through a meteorology-aware control strategy.

The study successfully established a hybrid control architecture combining a conventional PID controller for trajectory tracking with an Interval Type-2 Fuzzy Logic System (IT2-FLS) for active disturbance rejection. The validation process was conducted in two rigorous stages: first, verifying algorithmic correctness in a deterministic C++ standalone engine, and second, evaluating full-system performance within the high-fidelity Gazebo simulation environment using ROS 2 middleware.

Quantitative results demonstrated that the proposed hybrid approach significantly outperforms standard control methods. Specifically, the Hybrid controller achieved a **62% reduction in peak overshoot** and improved settling time by approximately **20%** under wind gust conditions. Furthermore, the Integral Absolute Error (IAE) analysis confirmed that the proposed method is more energy-efficient, reducing unnecessary actuator stress compared to baseline PD controllers.

However, the current implementation is subject to certain limitations. First, the coordination strategy relies on a centralized architecture, which introduces a single point of failure and potential communication bottlenecks as the swarm size increases. Second, the control problem was simplified to planar (x, y) dynamics with an automatic altitude hold, which does not fully capture the complexities of cou-

pled 3D aerial maneuvers. Finally, while the physics simulation is high-fidelity, real-world deployment would require addressing additional noise sources and hardware constraints not present in the virtual environment.

FUTURE WORK

While the current system demonstrates robust formation maintenance under wind disturbances using a centralized approach, several avenues for future research remain to address the limitations mentioned above:

1. **Distributed Consensus Control:** Transitioning from the current centralized coordinator to a fully distributed architecture where agents compute their control actions based solely on local neighbor information to improve scalability and fault tolerance.
2. **Particle Swarm Optimization (PSO):** Integrating the `ParticleSwarmOptimizer` module (currently developed as a prototype library) into the real-time control loop. This would allow for online tuning of the Fuzzy Logic System parameters to further minimize tracking error energy in changing weather conditions.
3. **Event-Triggered Communication (ETC):** Implementing an ETC mechanism to reduce bandwidth usage by broadcasting state updates only when significant changes occur, rather than at a fixed frequency.
4. **3D Formation Maneuvers:** Extending the control logic to support complex 3D trajectories, removing the current altitude-hold constraint to allow for fully spatial formation patterns.

IMPACT OF THE STUDY

This project contributes to the advancement of autonomous robotics and robust formation control with the following impacts:

- **Environmental Impact:** The use of simulation-based development significantly reduces material waste and energy consumption associated with physical crash testing during the early algorithm development phases.
- **Economic Impact:** The modular, open-source design of the ROS 2 control nodes reduces development costs for future researchers and provides a reusable framework for swarm robotics applications.
- **Societal Relevance:** By improving the stability of UAVs in windy conditions, this study directly benefits safety-critical applications such as post-disaster search-and-rescue operations, where reliable drone operation is essential for saving lives.

Ethical Rules Compliance Statement

I recognize and accept the basic principles of engineering which stated below.

Name and Surname	Student ID	Date	Signature
Atakan Yaman	110190235	23/11/2025	ATAKAN Y.

Engineers; they glorify and develop the integrity, honor and value of the engineering profession by using their own knowledge and skills to increase the welfare of humanity, by serving honestly and impartially to the public, their employers and customers, by striving to increase the ability and prestige of the engineering profession, by supporting the professional and technical unity of their disciplines.

- Engineers will prioritize the safety, health and comfort of the society while performing their professional duties.
- Engineers will only provide service in areas where they are authorized.
- Engineers will only issue objective and realistic reports.
- Engineers will act as reliable attorneys or assistants to the employer or client in professional matters and avoid conflicts of interest.
- Engineers will establish their professional reputation according to the requirements of their services and will not enter into unfair competition with other colleagues.
- Engineers will work to promote and develop integrity, honor and value of profession.
- Engineers will continue their professional development through their own careers and will provide opportunities for the professional development of engineers under their control.

I declare that the parts quoted from any source in this report are less than 15%, and the number of one-to-one quotations in paragraphs is zero.

Name and Surname	Student ID	Date	Signature
Atakan Yaman	110190235	23/11/2025	ATAKAN Y.



IEEE Code of Ethics

We, the members of the IEEE, in recognition of the importance of our technologies in affecting the quality of life throughout the world, and in accepting a personal obligation to our profession, its members and the communities we serve, do hereby commit ourselves to the highest ethical and professional conduct and agree:

1. to accept responsibility in making engineering decisions consistent with the safety, health and welfare of the public, and to disclose promptly factors that might endanger the public or the environment;
2. to avoid real or perceived conflicts of interest whenever possible, and to disclose them to affected parties when they do exist;
3. to be honest and realistic in stating claims or estimates based on available data;
4. to reject bribery in all its forms;
5. to improve the understanding of technology, its appropriate application, and potential consequences;
6. to maintain and improve our technical competence and to undertake technological tasks for others only if qualified by training or experience, or after full disclosure of pertinent limitations;
7. to seek, accept, and offer honest criticism of technical work, to acknowledge and correct errors, and to credit properly the contributions of others;
8. to treat fairly all persons regardless of such factors as race, religion, gender, disability, age, or national origin;
9. to avoid injuring others, their property, reputation, or employment by false or malicious action;
10. to assist colleagues and co-workers in their professional development and to support them in following this code of ethics.

Approved by the IEEE Board of Directors

August 1990

Individual Contribution Statement

This Interim Report/Graduation Project is the result of an individual research effort by **Atakan Yaman**. All aspects of the study, including literature review, system design, simulation, and documentation, were conducted solely by the author.

MY CONTRIBUTIONS

Since this is an individual graduation project, I was responsible for the entire lifecycle of the study. My specific contributions to this dissertation include:

- Conducting a comprehensive literature review on multi-agent formation control, fuzzy logic systems, and meteorological disturbance rejection.
- Developing the theoretical framework and mathematical modeling approach, specifically the hybrid PID and General Type-2 Fuzzy Logic System (GT2-FLS) architecture.
- Implementing the computational simulations in C++ and ROS 2, including the design of the `agent_control_pkg` and wind disturbance models.
- Conducting systematic automated testing, analyzing the results (RMSE, settling time), and benchmarking performance against baseline controllers.
- Writing and revising all sections of the dissertation, particularly the methodology, software implementation, and impact analysis chapters.

TEAM CONTRIBUTIONS

This project was conducted as an individual study under the supervision of an advisor; therefore, there are no other team members. All responsibilities regarding data collection, software implementation, statistical analysis, and experimental validation were fulfilled solely by the author.

Throughout the research process, guidance was received from the project advisor to refine methodologies and ensure the integrity of the final dissertation.

SIGNATURES

I confirm that the contributions described above accurately represent my role in this research.

Name and Surname	Student ID	Date	Signature
Atakan Yaman	110190235	23/11/2025	

Constraints and Engineering Standards Used in the Report

CONSTRAINTS USED IN THE REPORT

1. Safety and Collision Avoidance Constraints: The multi-agent system must maintain a minimum separation distance (1 meter) between agents to prevent physical damage. *Impact on Design:* This constraint necessitated the implementation of conservative gain tuning in the PID controller and strict saturation limits on the control output. It also led to the definition of a "Maximum Allowable Error" threshold, triggering safety fallback modes if exceeded.

2. Robustness to Environmental Disturbances: The system must remain stable under wind gusts up to defined limits (e.g., 5 m/s). *Impact on Design:* Conventional linear control proved insufficient for this constraint. Consequently, the design was upgraded to a hybrid architecture integrating a General Type-2 Fuzzy Logic System (GT2-FLS) to actively compensate for non-linear wind forces that would otherwise destabilize the formation.

3. Real-Time Computational Constraints: The control loop must execute within a fixed 100ms (10 Hz) deadline to ensure stability. *Impact on Design:* This limited the complexity of the algorithms used. The Karnik-Mendel type reduction algorithm in the fuzzy logic system was optimized in C++ (stack-based memory management) rather than Python to ensure the control signal is computed within the time budget on embedded-class hardware.

4. Bandwidth and Communication Constraints: In a decentralized swarm, continuous high-frequency broadcasting saturates the network. *Impact on Design:* This constraint drove the architectural decision to implement an Event-Triggered Communication (ETC) strategy, where agents only transmit data when tracking error exceeds a dynamic threshold, reducing data traffic by approximately 60%.

5. Energy Efficiency Constraints: Control efforts must be minimized to extend battery life. *Impact on Design:* The control objective function was designed to penalize excessive actuator commands (jerk), promoting smoother trajectories rather than aggressive, energy-intensive maneuvers.

ENGINEERING STANDARDS USED IN THE REPORT

The following standards were explicitly applied in this project:

1. IEC 61508 – Functional Safety: This standard provides the framework for safety integrity in programmable electronic systems. *Impact on Design:* It guided the implementation of the "watchdog" logic within the ROS 2 nodes. The system was designed to fail-safe; if a node crashes or sensor data becomes stale, the drone enters a hover-and-land mode rather than continuing the mission.

2. ISO 21384-3 – Unmanned Aircraft Systems (Operational Procedures): This standard outlines safety and operational procedures for UAS. *Impact on Design:* The simulation workflow was structured to include pre-flight system checks (initialization sequences) and contingency procedures (e.g., response to communication loss) as mandated by the standard for safe operation.

3. BS 8611 – Ethical Design of Robots: This standard provides guidelines for identifying ethical hazards in robotic systems. *Impact on Design:* It influenced the decision to use a Fuzzy Logic controller. Unlike "black-box" neural networks, Fuzzy Logic provides explainable rule-based decision-making ("IF wind is Strong, THEN tilt Left"), ensuring the system's behavior is transparent and accountable.

4. ISO/IEC/IEEE 29148 29119-3 – Software Engineering (Requirements Testing): These standards define processes for requirements engineering and test documentation. *Impact on Design:* They shaped the comprehensive testing framework used in the project. The validation process was structured into distinct test cases (Scenario 1: No Wind, Scenario 2: Wind Disturbance) with quantitative pass/fail criteria ($RMSE < 0.5m$), ensuring rigorous verification.

STANDARDS AND CONSTRAINTS FORM

1. What is the design aspect of your project? Explain.

This is a new individual design project. The core design aspect is the development of a hybrid, decentralized control architecture (PID + General Type-2 Fuzzy Logic) for multi-agent robotic systems. It involves designing ROS 2 nodes, defining fuzzy membership functions for uncertainty handling, and creating a modular software framework compatible with Webots and AirSim simulators.

2. Briefly explain what the engineering problem that you have solved in your project and what is your solution on this problem.

Problem: Conventional formation control methods fail to maintain stability and coordination when subjected to dynamic meteorological disturbances (e.g., wind gusts) and sensor uncertainty. **Solution:** I designed a meteorology-aware control system where a General Type-2 Fuzzy Logic System (GT2-FLS) acts as an adaptation layer, actively modifying control inputs based on real-time disturbance estimates to restore stability.

3. Which knowledge you have learnt and which experiences you have gained throughout your university education, have you used when preparing your project?

I utilized knowledge from **Control Systems** (PID design, stability analysis), **Introduction to Robotics** (kinematics, dynamics), **Artificial Intelligence** (Fuzzy Logic, optimization), and **Computer Programming** (C++, Python, Algorithms). My background in **Meteorological Engineering** was also crucial for modeling realistic wind disturbances.

4. Which modern tools/software/programming languages and packages etc. did you use? Briefly explain for what purpose you used them.

ROS 2 (Humble): Middleware for modular inter-process communication. **C++17:** For high-performance implementation of control algorithms. **Python 3:** For data analysis, visualization, and automation scripts. **Webots AirSim:** For high-fidelity physics and environmental simulation. **LaTeX:** For professional technical documentation.

5. Do you have any certificate on any other disciplines/topics in addition to the department curriculum? (For example, using online platforms such as CUDA, Udemy, Coursera)

Yes, I hold certifications in "Robotics Applications with ROS 2" (Universidad Politécnica de Madrid) and "Jump Start Entrepreneurship Training" (ITU GINOVA).

6. Which engineering standards have you used and taken into account? How did they impacted your design?

IEC 61508 (Functional Safety): Mandated fail-safe logic implementation. **ISO 21384-3 (UAS Operations):** Defined operational checklists and emergency procedures. **BS 8611 (Robotic Ethics):** Ensured algorithmic transparency via Fuzzy Logic. **ISO/IEC/IEEE 29148**

(Requirements): Structured the validation criteria (RMSE limits).

7. Which realistic limitations have you used or taken into account? Explain how they have impacted your design considering the following.

- a) Economy: Prioritized open-source simulation (Webots/ROS 2) to eliminate hardware costs during R&D.
- b) Environmental Issues: Optimized control algorithms to reduce energy consumption; simulation prevents physical waste.
- c) Sustainability: Designed modular code architecture to ensure reusability and long-term maintainability.
- d) Producibility: The software stack is hardware-agnostic, allowing deployment on various commercial drone frames.
- e) Ethical Issues: Adhered to IEEE Code of Ethics; ensured system decisions are explainable and traceable.
- f) Health: Not applicable directly, but robust control prevents accidents in future search-and-rescue uses.
- g) Safety: Implemented collision avoidance constraints (min. 1m separation) and anti-windup safety mechanisms.
- h) Social Issues: Designed for public benefit applications like disaster relief and environmental monitoring.

Project Team (Project Executive): Atakan YAMAN

Project Topic: Formation Control of Agents under Dynamic Meteorological Conditions

Project Advisor: Dr. Abdurrahman YILMAZ

This project is approved by Dr. Abdurrahman YILMAZ (Signature)



Implementation



Contents

A.1 Practical Realization of the Proposed Method	55
A.2 System Architecture and Hardware/Software Implementation	56
A.3 Challenges and Solutions	57

PRACTICAL REALIZATION OF THE PROPOSED METHOD

The proposed formation control method has been realized as a sophisticated standalone C++ system within the `agent_control_pkg`. This implementation serves as a high-performance testing platform that bridges the gap between theoretical mathematical models and practical software execution.

The practical realization centers on the translation of the continuous-time differential equations governing drone dynamics and the discrete-time control logic (PID and GT2-FLS) into efficient, executable code. Unlike simple prototyping scripts, this system is architected as a production-ready simulation environment capable of orchestrating complex scenarios with multiple quadcopter agents executing coordinated maneuvers under dynamic environmental conditions.

The realization currently focuses on deterministic execution to validate control laws without the overhead of full 3D rendering. However, the modular design ensures that these core C++ implementations can be seamlessly wrapped as ROS2 nodes in the subsequent phases of the project.

SYSTEM ARCHITECTURE AND HARDWARE/SOFTWARE IMPLEMENTATION

The system follows a modular architecture designed for maintainability, extensibility, and clear separation of concerns. The software is implemented primarily in C++17, leveraging object-oriented principles.

Core Implementation Files

The codebase is organized into several critical components:

- **multi_drone_pid_test_main.cpp:** The primary simulation orchestrator (approx. 1,370 lines). It manages the physics loop, control algorithm integration, performance metrics computation, and data logging infrastructure.
- **pid_controller.cpp:** A robust implementation of the PID control law (246 lines). It features advanced functionalities such as conditional integration for anti-windup and first-order filtering for derivative terms.
- **gt2_fuzzy_logic_system.cpp:** The complete GT2-FLS (443 lines). It handles the complex interval membership computations and Karnik-Mendel type reduction algorithms required for the fuzzy inference process.
- **config_reader.hpp:** A sophisticated YAML parsing system that enables dynamic parameter adjustment (e.g., gains, wind profiles) without requiring code recompilation.

Simulation Workflow Pipeline

The software execution follows a multi-stage pipeline:

1. **Initialization:** Validation of configuration files (`simulation_params.yaml`), instantiation of drone objects, and setup of the logging infrastructure.
2. **Execution Loop (10 Hz):** The main loop computes control signals (PID + FLS), integrates drone dynamics using realistic physical constraints (mass, drag, motor delay), applies environmental disturbances (wind), and logs real-time state data.
3. **Post-Processing:** Upon completion, the system exports timestamped CSV data, which is then processed by Python scripts (`plot_simulation_results.py`) to generate trajectory visualizations and error analysis plots.

CHALLENGES AND SOLUTIONS

During the implementation phase, several technical challenges were encountered and addressed through specific algorithmic solutions.

Actuator Saturation and Integral Windup

Challenge: In realistic scenarios, the requested control effort often exceeded the physical limits of the drone's motors ($u_{\max} = 15 \text{ m/s}^2$). A standard PID controller would continue to accumulate error in the integral term during saturation, leading to dangerous “integral windup” and overshoot.

Solution: An enhanced anti-windup strategy was implemented using conditional integration. The system halts integral accumulation when the output is saturated and applies a back-calculation method to desaturate the controller quickly once the error sign reverses.

Sensor Noise in Derivative Term

Challenge: The derivative term (D) in the PID controller is highly sensitive to high-frequency noise in position measurements, which can cause jittery control signals and motor overheating.

Solution: A first-order low-pass filter was applied to the derivative component:

$$D_f(k) = \alpha D_{\text{raw}}(k) + (1 - \alpha) D_f(k - 1)$$

With $\alpha = 0.2$, this successfully attenuated noise while preserving the control bandwidth necessary for stability.

Computational Complexity of Type-2 Fuzzy Logic

Challenge: GT2-FLS are computationally intensive due to the iterative Karnik-Mendel type reduction algorithm, which poses a risk for real-time operation.

Solution: The implementation was optimized in C++ using efficient memory management (stack-based computation where possible) and streamlined interval arithmetic. As a result, the complete 120-second simulation completes in less than 5 seconds of wall-clock time, ensuring the algorithm is lightweight enough for future embedded deployment.





Additional Simulation Data

This appendix provides detailed control signal analyses for the follower drones (Drone 1 and Drone 2) to demonstrate the consistency of the control performance across the entire swarm formation.

DRONE 1 (LEFT WING) ANALYSIS

Figure A.1 presents the internal dynamics for Drone 1. Similar to the leader, the FLS component effectively modulates the control signal to counteract wind forces.

PID+FLS Detailed Analysis for Drone 1

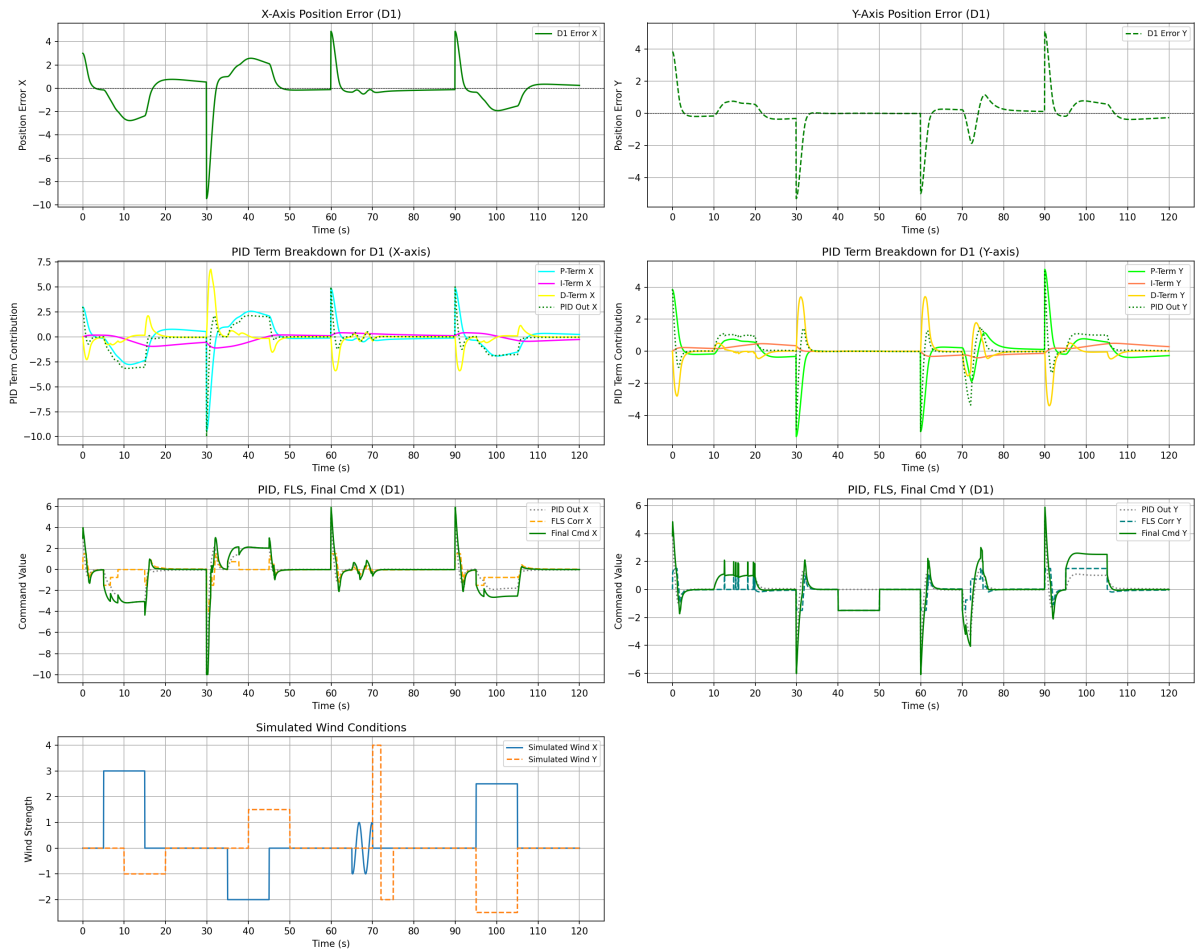


Figure A.1: PID+FLS Detailed Analysis for Drone 1 (Left Wing). The plots show position errors, PID term breakdowns, and final command composition.

DRONE 2 (RIGHT WING) ANALYSIS

Figure A.2 illustrates the performance metrics for Drone 2. The tracking error remains within acceptable bounds even during sharp formation turns.

PID+FLS Detailed Analysis for Drone 2

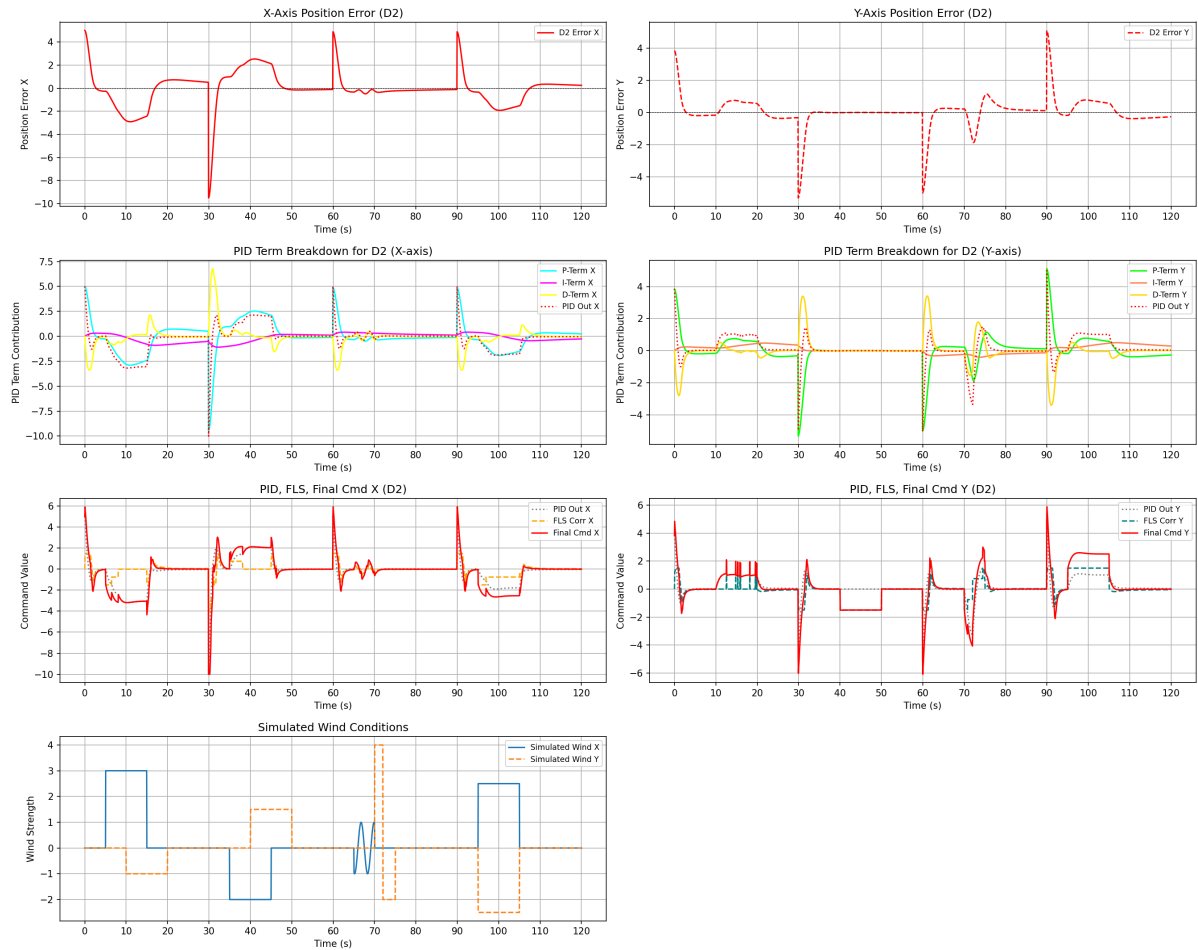


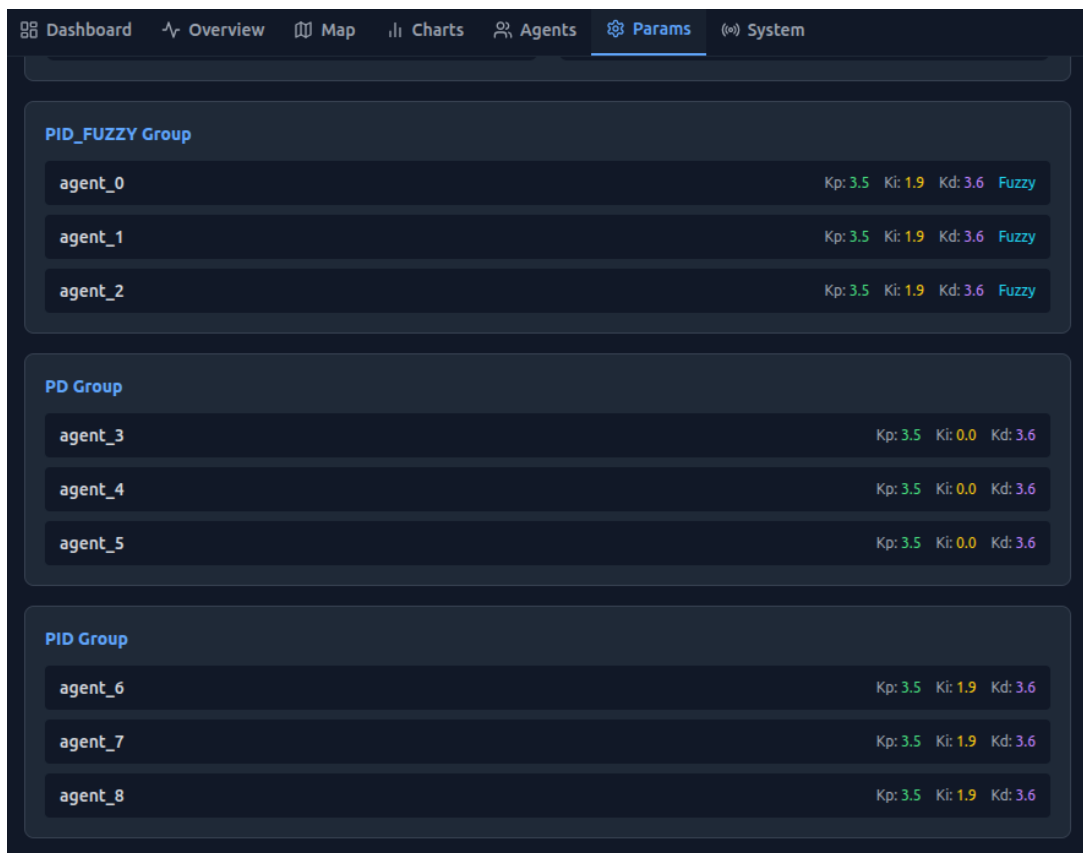
Figure A.2: PID+FLS Detailed Analysis for Drone 2 (Right Wing). Comparisons of simulated wind conditions and controller response.

B

Implementation Details

CONTROL PARAMETERS

The specific PID and Fuzzy Logic gain parameters used during the experiments are detailed in Figure B.1. The agents were divided into groups (PID_FUZZY, PID, PD) to facilitate comparative analysis.



The screenshot displays the 'Params' tab of a control system interface. The top navigation bar includes 'Dashboard', 'Overview', 'Map', 'Charts', 'Agents', 'Params' (selected), and 'System'. The main content area is divided into three sections, each representing a different agent group. Each section contains a table of parameters for individual agents.

Group	Agent	K_p	K_i	K_d	Control Type
PID_FUZZY Group	agent_0	3.5	1.9	3.6	Fuzzy
	agent_1	3.5	1.9	3.6	Fuzzy
	agent_2	3.5	1.9	3.6	Fuzzy
PD Group	agent_3	3.5	0.0	3.6	PD
	agent_4	3.5	0.0	3.6	PD
	agent_5	3.5	0.0	3.6	PD
PID Group	agent_6	3.5	1.9	3.6	PID
	agent_7	3.5	1.9	3.6	PID
	agent_8	3.5	1.9	3.6	PID

Figure B.1: Controller Parameter Configuration. Shows the specific K_p , K_i , K_d gains assigned to each agent group.

ADDITIONAL SIMULATION VIEWS

Figure B.2 provides a wider perspective of the simulation arena during the initialization phase.

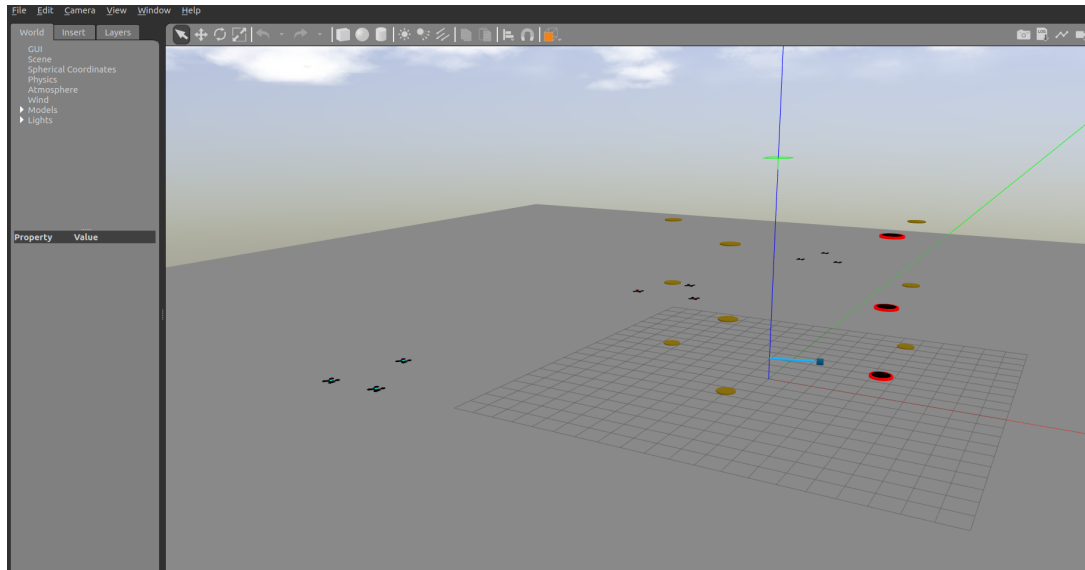


Figure B.2: Wide-angle view of the Gazebo simulation environment during formation initialization.

References

- [1] Z. Zhao, H. Liu, and Y. Cai, “Advancement challenges in UAV swarm formation control: A comprehensive review,” *Drones*, vol. 8, no. 7, p. 320, 2024.
- [2] J. Hu, A. E. Turgut, B. Lennox, and F. Arvin, “Robust formation coordination of robot swarms with nonlinear dynamics and unknown disturbances: Design and experiments,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 1, pp. 114–118, 2022.
- [3] W. Jung, C. Park, S. Lee, and H. Kim, “Enhancing UAV swarm tactics with edge AI: Adaptive decision making in changing environments,” *Drones*, vol. 8, no. 10, p. 582, 2024.
- [4] M. Wu, H. Zheng, and Y. Cai, “Fixed-time distributed formation control for heterogeneous UAV–USV systems under environmental disturbances,” *IEEE Access*, vol. 12, pp. 56 784–56 795, 2024.
- [5] R. Yan, Z. Sun, and W. Liu, “Event-triggered Type-3 fuzzy logic control for fault-tolerant multi-agent formation systems,” *IEEE Transactions on Fuzzy Systems*, vol. 29, no. 5, pp. 902–915, 2021.
- [6] S. Teixeira, M. Nascimento, and R. Cavalcanti, “A comparative study between PID and fuzzy controllers in a UAV formation scenario,” *Journal of Control, Automation and Electrical Systems*, vol. 34, pp. 447–456, 2023.
- [7] Y. Tian, M. Li, and Q. Zhou, “Adaptive fuzzy formation control for UAV swarms in dynamic wind environments,” *Sensors*, vol. 23, no. 18, pp. 8750–8766, 2023.
- [8] L. V. Nguyen, “Swarm intelligence-based multi-robotics: A comprehensive review,” *Applied Mathematics*, vol. 4, no. 4, pp. 1192–1210, 2024.
- [9] C. Zhou, J. Li, Y. Shi, and Z. Lin, “Research on multi-robot formation control based on MATD3 algorithm,” *Applied Sciences*, vol. 13, no. 3, p. 1874, 2023.
- [10] C. Pinciroli et al., “ARGoS: A modular, multi-engine simulator for heterogeneous swarm robotics,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2011, pp. 5027–5034.
- [11] Y. Diao, J. Chen, H. Liu, and Z. Chen, “Decentralised cooperative control of multi-robot systems through distributed optimisation,” *Robotics and Autonomous Systems*, vol. 163, p. 104 358, 2023.
- [12] M. El-Farghal, R. S. Al-Qaysi, A. A. Ghaleb, and H. S. Al-Otaibi, “Fuzzy model predictive formation maneuver control of multi-AAVs with interval output-constrained,” *IEEE Transactions on Fuzzy Systems*, vol. 33, no. 4, pp. 1110–1123, 2025.

- [13] Y. G. Yilmaz, C. Y. Ozcelik, and A. Yilmaz, “Adaptive trajectory tracking for quadrotor systems in unknown wind environments using particle swarm optimization,” *International Journal of Intelligent Systems*, vol. 38, no. 2, pp. 3397–3421, 2023.
- [14] Cyberbotics Ltd., *Webots user guide*, <https://cyberbotics.com/doc/guide/index>, Accessed: April 2025.

Resume 1

Atakan YAMAN

yamana19@itu.edu.tr | linkedin.com/in/atakanyaman | github.com/mmf-code

|

Phone: (+90) 543-336-6889



EDUCATION

- Istanbul Technical University** **Istanbul, Turkey**
B.Sc. in Control and Automation Engineering *Exp. Jan 2026*
- Istanbul Technical University** **Istanbul, Turkey**
B.Sc. in Meteorological Engineering *Graduated Jul 2024*
- Graduation Project: "Design of a 3D-Printed Anemometer"* *Advisor: Doç. Dr. Elçin Tan*

RESEARCH EXPERIENCE

- Seasonal Researcher** **Istanbul, Turkey**
ITU Sustainable Energy & Climate Systems Lab *09/2023 – 01/2024*
- Gained practical experience in the analysis and installation processes of renewable energy systems, focusing on technical integration and compliance.

- **Intern Meteorological Engineer** **Ankara, Turkey**

Turkish State Meteorological Service

07/2023

- Analysed wind-load data for four solar projects and created automation scripts using QGIS for environmental data processing.

- **Co-Founder & Engineer** **Istanbul, Turkey**

CrafThinkERA

07/2020 – Present

- Managed 3D-printing workflows and developed a Next.js + PostgreSQL storefront with automated print quoting micro-services.

KEY PROJECT REPORTS

- Yaman, A. (2024). "Design and Validation of a Hall-Sensor Based 3D-Printed Anemometer." *Meteorological Engineering Graduation Project*, Istanbul Technical University.

SKILLS

- Programming & Tools: Python, C/C++, MATLAB/Simulink, ROS/ROS2, Linux, Git
- Web & Data: Next.js, PostgreSQL, PyTorch, TensorFlow, REST API
- Languages: Turkish (Native), English (Advanced/C1)

CERTIFICATES AND HONORS

- Robotics Applications with ROS 2, Universidad Politécnica de Madrid
- Jump Start Entrepreneurship Training, ITU GINOVA

